R Course: Beginner to Expert

Sri Ram

2025-11-14

Table of contents

1 Welcome **8**

2 About this Course **9**
2.1 How to Use 9

3 R Basics **10**

4 R as a Calculator **11**

5 Objects & Assignment **12**

6 object naming rules **13**

7 Basic Operations in R **14**

8 Comments in R **16**

9 Getting Help in R **17**

10 Installing and Loading Packages in R **19**
10.0.1 Saving and Loading Workspaces in R 19

11 Working Directory **20**

12 Vectors (Atomic) **21**

13 Exercises **22**

14 Data Types & Data Structures **23**

15 Vectors **24**

16 Lists **38**

17 Matrices **50**

18 Data Frames **71**

19 Factors **80**

20 Arrays **82**

21 Summary **83**

22 Manipulating Vectors, Data Frames, and Lists **84**

23 Vectors: Indexing & Vectorized Ops **85**

24 Data Frames with dplyr **104**

25 Read sas dataset **118**

26 keeping selected columns **119**

27 dropping columns **120**

28 Rename **122**

29 Arrange (sorting) **123**

30 convert var names to lower case **124**

31 Creating new variables **125**

32 bind rows (set operator in SAS) **126**

33 recode (similar to PROC FORMAT in SAS) **127**

34 joins **128**
34.1 innter join 129
34.2 left join 129
34.3 right join 130
34.4 Full Join 130
34.5 Semi Join 130
34.6 Anti join 131

35 Reshaping data with tidyr **133**

36 counting with `n()` **134**

37 dplyr distinct() **136**

38 case_when() **137**

39 create seq no. **138**

40 Lists: `lapply`, purrr **139**

41 Exercises **140**

42 String Functions **141**

43 String Functions in Base R **142**
43.1 Basic String Functions . . . 142
43.2 Pattern Matching and Replacement . . . 143
43.3 String Splitting and Joining . . . 144
43.4 Regular Expressions . . . 144
43.5 Example Usage . . . 146

44 Overall differences **147**

45 Detect matches **150**
45.1 `str_detect()`: Detect the presence or absence of a pattern in a string . . . 150
45.2 `str_which()`: Find positions matching a pattern . . . 150
45.3 `str_count()`: Count the number of matches in a string . . . 151
45.4 `str_locate()`: Locate the position of patterns in a string . . . 151
45.5 `str_subset()`: Keep strings matching a pattern, or find positions . . . 152
45.6 `str_extract()`: Extract matching patterns from a string . . . 153
45.7 `str_match()`: Extract matched groups from a string . . . 154

46 Manage lengths **156**
46.1 `str_length()`: The length of a string . . . 156
46.2 `str_pad()`: Pad a string . . . 157
46.3 `str_trunc()`: Truncate a character string . . . 158
46.4 `str_trim()`: Trim whitespace from a string . . . 158
46.5 `str_wrap()`: Wrap strings into nicely formatted paragraphs . . . 159

47 Mutate strings **161**
47.1 `str_replace()`: Replace matched patterns in a string . . . 161
47.2 case: Convert case of a string . . . 161

48 Join and split **164**
48.1 `str_flatten()`: Flatten a string . . . 164
48.2 `str_dup()`: duplicate strings within a character vector . . . 164
48.3 `str_split()`: Split up a string into pieces . . . 165
48.4 `str_glue()`: Interpolate strings . . . 166

49 Order strings **168**
49.1 `str_order()`: Order or sort a character vector . . . 168

50 date and time functions **172**
50.0.1 Date and Time Base Functions in R . . . 172
50.0.2 Creating Date and Time Objects . . . 172
50.0.3 Year Formats . . . 172
50.0.4 Month Formats . . . 172
50.0.5 Day Formats . . . 173
50.0.6 Weekday Formats . . . 173
50.0.7 Time Formats . . . 173
50.1 1. Date & Time Basics in R . . . 177
50.1.1 1.1 Core classes . . . 177
50.1.2 1.2 Converting numerics to Date (origins matter) . . . 177
50.1.3 1.3 Custom date formats . . . 178
50.1.4 1.4 Datetime strings (ISO 8601 & time zones) . . . 178
50.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic . . . 178
50.2.1 2.1 Intuitive parsers . . . 178
50.2.2 2.2 Accessors and rounding . . . 179
50.2.3 2.3 Timespans . . . 179
50.2.4 3 ADaM-Style Partial Date Imputation (Start & End) . . . 179
50.2.5 Start date (`AESTDTC`) . . . 179
50.2.6 End date (`AEENDTC`) . . . 180
50.2.7 Censoring by death / last alive date . . . 180
50.2.8 Imputation flags . . . 180
50.3 4. Durations & Analysis Variables . . . 182
50.3.1 4.1 AE duration (AEDUR) in days . . . 182

51 Using difftime (base) or as.numeric on Date differences **183**
51.0.1 6.2 Handy lubridate parsers (subset) . . . 183

52 Reading SAS Datasets (+ Cleaning) **184**
52.1 Handling Labels & Missing . . . 186
52.2 Labelled to Factor (if needed) . . . 186
52.3 Common Cleaning . . . 186

53 Exercises **188**

54 Base R Functions & Apply Family **189**

55 Common Utilities **190**

56 Apply Family **191**

57 Subsetting Essentials **192**

58 Exercises **193**

59 Custom Functions & Validation **194**

60 Writing Functions **195**

61 Error Handling **196**

62 Unit Testing with testthat **197**

63 Document with roxygen2 **198**

64 Exercises **199**

65 R Package Development **200**
65.1 Setup . . . 200
65.2 Create a Package . . . 200
65.3 Add a Function . . . 200
65.4 Build, Install, Check . . . 200
65.5 Vignette & Website . . . 201

66 Git in RStudio (Setup & Auth) **202**
66.1 One-Time Setup . . . 202
66.2 Initialize Git for the Current Project . . . 202
66.3 Connect to GitHub . . . 202
66.4 Typical Workflow . . . 202
66.5 Remove Git from a Project (macOS/RStudio) . . . 203

67 Creating ADaM: ADSL from SDTM-like Inputs **204**
67.1 Build ADSL . . . 205

68 TLFs: Table, Figure, Listing **207**
68.1 Table 1: Baseline Characteristics by Treatment . . . 208
68.2 Figure: (Toy) Survival Curve . . . 208
68.3 Listing: Subject-Level Listing . . . 209

69 Capstone: End-to-End Mini Workflow **211**
69.1 Parameters . . . 211
69.2 1) Read (or Synthesize) SDTM . . . 211
69.3 2) Derive ADSL (Minimal Demo) . . . 212
69.4 3) TLFs . . . 213
69.5 4) Save Outputs . . . 214

70 Appendix: Tips, Profiles, .libPaths **215**
70.1 Useful Profiles 215
70.2 Custom Library Paths 215
70.3 Format vs formatC (quick recap) 215
70.4 POSIXct vs POSIXlt 216
70.5 Recommended Packages 216
70.6 Short Glossary 216

1 Welcome

2 About this Course

This Course from **R beginner** to **expert**, with a practical focus on **clinical programming** (CDISC/ADaM and TLFs).
Each chapter includes step-by-step explanations, runnable code, and short exercises.

2.1 How to Use

1. Install: R (4.2), RStudio (or VS Code).
2. Follow along chapter-by-chapter, running code and completing exercises.
3. Use sample data or your own clinical trial datasets (SAS/CSV). ## Structure (Highlights)

- **Basics & Data**: R syntax, data types/structures, vectors/data frames/lists.
- **I/O**: Read SAS datasets (with `haven`), handle labels, and clean raw data.
- **Programming**: Base functions, write your own functions, validate with tests.
- **DevOps**: Create an R package, connect Git in RStudio/GitHub.
- **CDISC**: Build ADaM (ADSL) from SDTM-like inputs.
- **TLFs**: Produce a baseline Table 1, a KM plot, and a listing.

  Tip: If you don't have sample SDTM/ADaM data yet, the chapters generate **small synthetic data** as a fallback so everything runs end-to-end. ## contact For questions or feedback, reach out to **r2sas2025@gmail.com**

3 R Basics

4 R as a Calculator

```
1 + 1
```

```
[1] 2
```

```
3 * (4 + 5)
```

```
[1] 27
```

5 Objects & Assignment

```
x <- 10
y <- 3.5
x + y
```

```
[1] 13.5
```

6 object naming rules

- R variable names can contain letters, numbers, periods, and underscores. However, they cannot start with a number or underscore. R is case-sensitive, so `age`, `Age`, and `AGE` would be considered different variables.
- R variable names should be descriptive and meaningful. Avoid using reserved words or function names as variable names.
- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). Rules for R variables are:
- A variable name must start with a letter and can be a combination of letters, digits, period(.) and underscore(_). If it starts with period(.), it cannot be followed by a digit.
- A variable name cannot start with a number or underscore (_) Variable names are case-sensitive (age, Age and AGE are three different variables) Reserved words cannot be used as variables (TRUE, FALSE, NULL, if…)
- Variable names should not contain spaces. Use underscore (_) or period (.) to separate words in a variable name.
- Variable names should be meaningful and descriptive. Avoid using single-letter variable names except for temporary variables in loops or functions.

7 Basic Operations in R

R supports various basic operations, including: * Arithmetic Operations: Addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (^). Example:

```
a <- 10
b <- 5
sum <- a + b
diff <- a - b
prod <- a * b
quot <- a / b
exp <- a ^ b
sum; diff; prod; quot; exp
```

```
[1] 15
```

```
[1] 5
```

```
[1] 50
```

```
[1] 2
```

```
[1] 1e+05
```

- Comparison Operations: Equal to (==), not equal to (!=), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). Example:

```
x <- 10
y <- 5
eq <- x == y
neq <- x != y
gt <- x > y
lt <- x < y
gte <- x >= y
lte <- x <= y
eq; neq; gt; lt; gte; lte
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

- Logical Operations: AND (`&`), OR (`|`), and NOT (`!`). Example:

```
p <- TRUE
q <- FALSE
and <- p & q
or <- p | q
not <- !p
and; or; not
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

8 Comments in R

Comments in R are created using the `#` symbol. Anything following the `#` on the same line is considered a comment and is ignored by R during execution. Example:

```
# This is a comment
x <- 10  # Assigning value to x
y <- 5   # Assigning value to y
sum <- x + y  # Calculating the sum of x and y
sum  # Output the sum
```

```
[1] 15
```

9 Getting Help in R

R provides several ways to get help and documentation for functions and packages: * `?function_name`: Displays the documentation for a specific function. Example:

```
?mean
```

- `help(function_name)`: Another way to access the documentation for a function. Example:

```
help(mean)
```

- `help.search("keyword")`: Searches for help topics related to a specific keyword. Example:

```
help.search("regression")
```

- `example(function_name)`: Shows examples of how to use a specific function. Example:

```
example(mean)
```

```
mean> x <- c(0:10, 50)

mean> xm <- mean(x)

mean> c(xm, mean(x, trim = 0.10))
[1] 8.75 5.50
```

- `vignette("package_name")`: Opens the vignette (detailed documentation) for a specific package. Example:

```
vignette("dplyr")
```

```
starting httpd help server ... done
```

- `??keyword`: Searches for help topics related to a specific keyword (similar to `help.search`). Example:

```
??regression
```

10 Installing and Loading Packages in R

R has a vast ecosystem of packages that extend its functionality. To use a package, you need to install it first and then load it into your R session. * Installing a Package: Use the `install.packages("package_name")` function to install a package from CRAN. Example:

```
install.packages("ggplot2")
```

- Loading a Package: Use the `library(package_name)` function to load an installed package into your R session. Example:

```
library(ggplot2)

# Now you can use functions from the ggplot2 package
```

10.0.1 Saving and Loading Workspaces in R

You can save your R workspace (all objects in memory) to a file and load it later * Saving Workspace: Use the `save.image("file_name.RData")` function to save the entire workspace to a file. Example:

```
save.image("my_workspace.RData")
```

- Loading Workspace: Use the `load("file_name.RData")` function to load a saved workspace from a file. Example:

```
load("my_workspace.RData")
```

11 Working Directory

```
getwd()
```

```
[1] "/home/runner/work/r4sas/r4sas"
```

```
# setwd("/path/you/want") # avoid in reproducible code; prefer here::here() for projects
```

12 Vectors (Atomic)

```r
nums <- c(1, 2, 3, 4)
chars <- c("a", "b", "c")
logical <- c(TRUE, FALSE, TRUE)
typeof(nums); typeof(chars); typeof(logical)
```

```
[1] "double"
```

```
[1] "character"
```

```
[1] "logical"
```

13 Exercises

1. Create an object `z` that stores `(5^2 + 7)/3`.
2. Use `?seq` and create a sequence from 0 to 1 by 0.1.
3. Inspect `typeof()` for a few objects you create.

14 Data Types & Data Structures

R has several built-in data structures to store and manipulate different types of data. These include vectors, lists, matrices, data frames, and factors. Below is an overview of each structure along with code examples.

15 Vectors

Vectors are the simplest data structure in R. They store elements of the same type (numeric, character, logical, etc.).

```
# Creating numeric and character vectors
numeric_vector <- c(1, 2, 3, 4)
char_vector1 <- c("apple", "banana", "cherry")
char_vector2 <- c(2, 3, 4, 5, "a")
logical_vector <- c(TRUE, FALSE, TRUE)

# Accessing elements
numeric_vector[1]  # Access the first element
```

```
[1] 1
```

```
v_logical <- c(T,F,T) # logical vector
v_logical
```

```
[1]  TRUE FALSE  TRUE
```

```
is.vector(v_logical)
```

```
[1] TRUE
```

```
is.atomic(v_logical)
```

```
[1] TRUE
```

```
typeof(v_logical)
```

```
[1] "logical"
```

```r
v_integer <- c(1L,2L,5L) # integer vector
v_integer
```

```
[1] 1 2 5
```

```r
is.vector(v_integer)
```

```
[1] TRUE
```

```r
is.atomic(v_integer)
```

```
[1] TRUE
```

```r
typeof(v_integer)
```

```
[1] "integer"
```

```r
v_double <- c(1.3,2.1,5.0) # double vector
v_double
```

```
[1] 1.3 2.1 5.0
```

```r
is.vector(v_double)
```

```
[1] TRUE
```

```r
is.atomic(v_double)
```

```
[1] TRUE
```

```r
typeof(v_double)
```

```
[1] "double"
```

```r
v_character <- c("a", "b", "c") # character vector
v_character
```

```
[1] "a" "b" "c"
```

```r
is.vector(v_character)
```

```
[1] TRUE
```

```r
is.atomic(v_character)
```

```
[1] TRUE
```

```r
typeof(v_character)
```

```
[1] "character"
```

```r
v_NULL <- NULL # NULL
v_NULL
```

```
NULL
```

```r
typeof(v_NULL)
```

```
[1] "NULL"
```

```r
# Mix type vector (type coercion or conversion)
v_mix <- c(T, 1L, 1.25, "a")
v_mix # all elements converted to charatcers (based on hierarchy)
```

```
[1] "TRUE" "1"    "1.25" "a"
```

```r
is.vector(v_mix)
```

```
[1] TRUE
```

```r
typeof(v_mix)
```

```
[1] "character"
```

```r
# Vector properties

v <- c(1,2,3,4,5)

# vector length
length(v)
```

```
[1] 5
```

```r
# type
typeof(v)
```

```
[1] "double"
```

```r
class(v)
```

```
[1] "numeric"
```

```r
# naming elements
names(v) # without names
```

```
NULL
```

```r
vnames <- c("first", "second", "third", "fourth", "fifth") # element names
names(v) <- vnames # naming elements
v
```

```
 first second  third fourth  fifth
     1      2      3      4      5
```

```r
names(v) # new names
```

```
[1] "first"  "second" "third"  "fourth" "fifth"
```

```r
# Create vector, access elements, modify vector

# create using c()
v <- c(1,3,5,8,0)

# create using operator :
1:100
```

```
  [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
 [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
 [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
 [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
 [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
 [91]  91  92  93  94  95  96  97  98  99 100
```

```r
10:-10
```

```
 [1]  10   9   8   7   6   5   4   3   2   1   0  -1  -2  -3  -4  -5  -6  -
7  -8
[20]  -9 -10
```

```r
# using sequence seq()
v <- seq(from = 1, to = 100, by = 1)
v
```

```
  [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
 [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
 [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
 [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
 [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
 [91]  91  92  93  94  95  96  97  98  99 100
```

```r
v <- seq(from = 0, to = 1, by = 0.01)
v
```

```
  [1] 0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10 0.11 0.12 0.13 0.14
 [16] 0.15 0.16 0.17 0.18 0.19 0.20 0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29
 [31] 0.30 0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.40 0.41 0.42 0.43 0.44
 [46] 0.45 0.46 0.47 0.48 0.49 0.50 0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59
 [61] 0.60 0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.70 0.71 0.72 0.73 0.74
 [76] 0.75 0.76 0.77 0.78 0.79 0.80 0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89
 [91] 0.90 0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.00
```

```r
v <- seq(from = 0, to = 10, length.out = 5)
v
```

```
[1]  0.0  2.5  5.0  7.5 10.0
```

```r
# let's create a vector for accessing vector elements
v <- 1:10
names(v) <- c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j")
v
```

```
 a  b  c  d  e  f  g  h  i  j
 1  2  3  4  5  6  7  8  9 10
```

```r
# access vector elements using integer vector index
v[c(1,5,10)]
```

```
 a  e  j
 1  5 10
```

```r
v[1:5] # range index selection (slicing)
```

```
a b c d e
1 2 3 4 5
```

```r
v[seq(from = 1, to = 9, by = 2)]
```

```
a c e g i
1 3 5 7 9
```

```r
v[10:1] # reverse order selection
```

```
 j  i  h  g  f  e  d  c  b  a
10  9  8  7  6  5  4  3  2  1
```

```r
v[c(10,1,5,3)] # mix orfer selection
```

```
 j  a  e  c
10  1  5  3
```

```
# access vector elements using logical vector index
v[c(T,F,F,F,F,F,F,F,F,F)] # access first element
```

```
a
1
```

```
v[c(F,F,F,F,F,F,F,T,T,T)] # access last three elements
```

```
 h  i  j
 8  9 10
```

```
# access elements using names
v[c("a","c","e")]
```

```
a c e
1 3 5
```

```
v[c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j")]
```

```
 a  b  c  d  e  f  g  h  i  j
 1  2  3  4  5  6  7  8  9 10
```

```
# modify vector elements
v
```

```
 a  b  c  d  e  f  g  h  i  j
 1  2  3  4  5  6  7  8  9 10
```

```
v[2] <- 20 # alter second element
v
```

```
 a  b  c  d  e  f  g  h  i  j
 1 20  3  4  5  6  7  8  9 10
```

```
v[c(1,5,10)] <- c(0,0,0) # alter multiple elements
v
```

```
 a  b  c  d  e  f  g  h  i  j
 0 20  3  4  0  6  7  8  9  0
```

```r
# modify elements with value 0
v
```

```
 a  b  c  d  e  f  g  h  i  j
 0 20  3  4  0  6  7  8  9  0
```

```r
v[v==0] # filter with condition
```

```
a e j
0 0 0
```

```r
v[v==0] <- 1000
v
```

```
   a    b    c    d    e    f    g    h    i    j
1000   20    3    4 1000    6    7    8    9 1000
```

```r
# truncate vector to first 3 elements
v <- v[1:3]
v
```

```
   a    b    c
1000   20    3
```

```r
# transpose vector change row to column vector or vice versa
v
```

```
   a    b    c
1000   20    3
```

```r
t(v)
```

```
        a  b c
[1,] 1000 20 3
```

```r
# delete or remove a vector
v <- NULL
v
```

```
NULL
```

```r
rm(v)

# combine 2 different vectors
v1 <- 1:3
v2 <- 100:105
v1
```

```
[1] 1 2 3
```

```r
v2
```

```
[1] 100 101 102 103 104 105
```

```r
v3 <- c(v1,v2) # combine vectors
v3
```

```
[1]   1   2   3 100 101 102 103 104 105
```

```r
# repet elements of a vector
rep(x = v1, times = 2)
```

```
[1] 1 2 3 1 2 3
```

```r
rep(x = v1, times = 100)
```

```
  [1] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
 [38] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
 [75] 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
[112] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[149] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[186] 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
[223] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[260] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[297] 3 1 2 3
```

```r
rep(10,10)
```

```
 [1] 10 10 10 10 10 10 10 10 10 10
```

```r
# Vector arithmetics

# vector - scalar (scalar with each vector element)
v <- 1:5
a <- 10
v
```

```
[1] 1 2 3 4 5
```

```r
a
```

```
[1] 10
```

```r
# Addition +
v + a
```

```
[1] 11 12 13 14 15
```

```r
# Subtraction -
v - a
```

```
[1] -9 -8 -7 -6 -5
```

```r
# Multiplication *
v * a
```

```
[1] 10 20 30 40 50
```

```r
# Division /
v / a
```

```
[1] 0.1 0.2 0.3 0.4 0.5
```

```r
# Exponent ^  **
v^a
```

```
[1]       1    1024   59049 1048576 9765625
```

```r
# Modulus (Remainder from division) %%
v %% 2
```

```
[1] 1 0 1 0 1
```

```r
# Integer Division %/%
v %/% 2
```

```
[1] 0 1 1 2 2
```

```r
# Other functions on vector elements
sqrt(v)
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.236068
```

```r
log(v)
```

```
[1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379
```

```r
sum(v)
```

```
[1] 15
```

```r
# vector - vector (vector element to element | member-by-member)
v1 <- seq(10,30,10)
v2 <- rep(3,3)

# Addition +
v1 + v2
```

```
[1] 13 23 33
```

```r
# Subtraction -
v1 - v2
```

```
[1]  7 17 27
```

```r
# Multiplication *
v1 * v2
```

```
[1] 30 60 90
```

```r
# Division /
v1 / v2
```

```
[1]  3.333333  6.666667 10.000000
```

```r
# Exponent ^  **
v1^v2
```

```
[1]  1000  8000 27000
```

```r
# Modulus (Remainder from division) %%
v1 %% v2
```

```
[1] 1 2 0
```

```r
# Integer Division %/%
v1 %/% v2
```

```
[1]  3  6 10
```

```r
# Vector-matrix style multiplication
v1
```

```
[1] 10 20 30
```

```r
v2
```

```
[1] 3 3 3
```

```r
10*3 + 20*3 + 30*3
```

```
[1] 180
```

```r
t(v1) %*% v2
```

```
     [,1]
[1,]  180
```

```r
v1 %*% v2
```

```
     [,1]
[1,]  180
```

```r
v1 %*% t(v2)
```

```
     [,1] [,2] [,3]
[1,]   30   30   30
[2,]   60   60   60
[3,]   90   90   90
```

```r
# Recycling rule
v1 <- c(1,1,1)
v2 <- 1:6
v1
```

```
[1] 1 1 1
```

```r
v2
```

```
[1] 1 2 3 4 5 6
```

```r
v1 + v2
```

```
[1] 2 3 4 5 6 7
```

```r
#  Set operations

v1 <- c("a", "b", "c")
v2 <- c("c", "d", "e")
v1
```

```
[1] "a" "b" "c"
```

```
v2
```

```
[1] "c" "d" "e"
```

```
union(v1,v2) # union of both sets (all unique elements)
```

```
[1] "a" "b" "c" "d" "e"
```

```
intersect(v1,v2) # intersection of both sets (elements in both sets)
```

```
[1] "c"
```

```
setdiff(v1,v2) # difference of elements (elements in v1 and not in v2)
```

```
[1] "a" "b"
```

```
identical(v1, v2) # check if vectors are identical
```

```
[1] FALSE
```

```
identical(c(1,2,3), c(1,2,3))
```

```
[1] TRUE
```

16 Lists

A list can contain elements of different types, including other lists or vectors or data structures.

```
# Creating a list
my_list <- list(name = "John", age = 25, scores = c(90, 85, 88))

# Accessing elements by name
my_list$name  # Output: "John"
```

```
[1] "John"
```

```
# Create a list (and name elements)

# lets create some variables (different types)
a <- 10
b <- 2L
c <- TRUE
d <- "word"
v <- 1:10
names(v) <- paste("i", v, sep = "")
M <- matrix(data = seq(10,40,by = 10), nrow = 2, dimnames = list(c("r1", "r2"), c("c1", "c2"))
A <- array(data = 1:8, dim = c(2,2,2), dimnames = list(c("r1", "r2"), c("c1", "c2"), c("M1",

# create list and include all variables (elements)
lst <- list(a, b, c, d, v, M, A)
lst
```

```
[[1]]
[1] 10

[[2]]
[1] 2

[[3]]
[1] TRUE
```

```
[[4]]
[1] "word"

[[5]]
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

[[6]]
   c1 c2
r1 10 30
r2 20 40

[[7]]
, , M1

   c1 c2
r1  1  3
r2  2  4

, , M2

   c1 c2
r1  5  7
r2  6  8
```

```
str(lst) # check list structure
```

```
List of 7
 $ : num 10
 $ : int 2
 $ : logi TRUE
 $ : chr "word"
 $ : Named int [1:10] 1 2 3 4 5 6 7 8 9 10
  ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ : num [1:2, 1:2] 10 20 30 40
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : chr [1:2] "r1" "r2"
  .. ..$ : chr [1:2] "c1" "c2"
 $ : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
  ..- attr(*, "dimnames")=List of 3
  .. ..$ : chr [1:2] "r1" "r2"
```

```
  .. ..$ : chr [1:2] "c1" "c2"
  .. ..$ : chr [1:2] "M1" "M2"
```

```
typeof(lst) # check type
```

```
[1] "list"
```

```
class(lst) # check class
```

```
[1] "list"
```

```
is.list(lst) # check if object is list
```

```
[1] TRUE
```

```
# name each list member
names(lst) <- c("a", "b", "c", "d", "v", "M", "A")
lst
```

```
$a
[1] 10

$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

$M
   c1 c2
r1 10 30
r2 20 40
```

```
$A
, , M1

   c1 c2
r1  1  3
r2  2  4

, , M2

   c1 c2
r1  5  7
r2  6  8
```

```
# alternative: define names as tags when list is created
list(a=a, b=b, c=c, d=d, v=v, M=M, A=A)
```

```
$a
[1] 10

$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

$M
   c1 c2
r1 10 30
r2 20 40

$A
, , M1

   c1 c2
```

```
r1  1  3
r2  2  4

, , M2

   c1 c2
r1  5  7
r2  6  8
```

```
# Access list elements

# single square bracket [] (return a list)
lst1 <-lst[1] # access first list elements (return a list)
str(lst1)
```

```
List of 1
 $ a: num 10
```

```
class(lst1)
```

```
[1] "list"
```

```
lst123 <-lst[c(1,2,3)] # access first three elements with index vector (return a list)
lst123
```

```
$a
[1] 10

$b
[1] 2

$c
[1] TRUE
```

```
class(lst123)
```

```
[1] "list"
```

```
# double square brackets [[]] (return original member)
ele <-lst[[5]] # extract 5th member-element (returns original element)
ele
```

```
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10
```

```
is.vector(ele)
```

```
[1] TRUE
```

```
# use $ operator - extract by member name (return original member)
ele <- lst$M
ele
```

```
   c1 c2
r1 10 30
r2 20 40
```

```
class(ele)
```

```
[1] "matrix" "array"
```

```
#  Modify list

# remove element from a list
lst
```

```
$a
[1] 10

$b
[1] 2

$c
[1] TRUE

$d
[1] "word"
```

```
$v
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

$M
   c1 c2
r1 10 30
r2 20 40

$A
, , M1

   c1 c2
r1  1  3
r2  2  4

, , M2

   c1 c2
r1  5  7
r2  6  8
```

```
lst[1] <- NULL # remove first member
lst
```

```
$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

$M
   c1 c2
r1 10 30
```

```
r2 20 40

$A
, , M1

   c1 c2
r1  1  3
r2  2  4

, , M2

   c1 c2
r1  5  7
r2  6  8
```

```
# add element to a list (at the end)
length(lst)
```

```
[1] 6
```

```
lst[7] <- 1000
lst
```

```
$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

$M
   c1 c2
r1 10 30
r2 20 40
```

```
$A
, , M1

   c1 c2
r1  1  3
r2  2  4

, , M2

   c1 c2
r1  5  7
r2  6  8


[[7]]
[1] 1000
```

```
# update value of a member in alist
lst[[7]] <- 500
lst[7]
```

```
[[1]]
[1] 500
```

```
# update value within a vector (on a list)
lst[[4]][5] <- 5000
lst[[4]]
```

```
  i1   i2   i3   i4   i5   i6   i7   i8   i9  i10
   1    2    3    4 5000    6    7    8    9   10
```

```
# convert list to a vector
vec <- unlist(lst)
vec
```

```
     b      c      d   v.i1   v.i2   v.i3   v.i4   v.i5   v.i6   v.i7   v.i8
   "2" "TRUE" "word"    "1"    "2"    "3"    "4" "5000"    "6"    "7"    "8"
  v.i9  v.i10     M1     M2     M3     M4     A1     A2     A3     A4     A5
   "9"   "10"   "10"   "20"   "30"   "40"    "1"    "2"    "3"    "4"    "5"
    A6     A7     A8
   "6"    "7"    "8"  "500"
```

```r
is.vector(vec)
```

```
[1] TRUE
```

```r
#  Merging lists & nested lists

# create another list
lst1 <- list(el1 = c(1,5,10), el2 = TRUE)

# merge both lists
lst_merged <- c(lst, lst1)
lst_merged
```

```
$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
  i1   i2   i3   i4   i5   i6   i7   i8   i9  i10
   1    2    3    4 5000    6    7    8    9   10

$M
   c1 c2
r1 10 30
r2 20 40

$A
, , M1

   c1 c2
r1  1  3
r2  2  4

, , M2

   c1 c2
```

```
r1  5  7
r2  6  8


[[7]]
[1] 500

$el1
[1]  1  5 10

$el2
[1] TRUE
```

```
str(lst_merged)
```

```
List of 9
 $ b  : int 2
 $ c  : logi TRUE
 $ d  : chr "word"
 $ v  : Named num [1:10] 1 2 3 4 5000 6 7 8 9 10
  ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ M  : num [1:2, 1:2] 10 20 30 40
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : chr [1:2] "r1" "r2"
  .. ..$ : chr [1:2] "c1" "c2"
 $ A  : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
  ..- attr(*, "dimnames")=List of 3
  .. ..$ : chr [1:2] "r1" "r2"
  .. ..$ : chr [1:2] "c1" "c2"
  .. ..$ : chr [1:2] "M1" "M2"
 $    : num 500
 $ el1: num [1:3] 1 5 10
 $ el2: logi TRUE
```

```
names(lst_merged)
```

```
[1] "b"   "c"   "d"   "v"   "M"   "A"   ""    "el1" "el2"
```

```
# nested list (recursive procedure)
list3 <- list(1, c(T,F,F)) # list sub-level 3
```

```r
list2 <- list(list3) # list sub-level 2
list1 <- list(list2) # list sub-level 1

str(list1)
```

```
List of 1
 $ :List of 1
  ..$ :List of 2
  .. ..$ : num 1
  .. ..$ : logi [1:3] TRUE FALSE FALSE
```

```r
# extract list level 2
list1[[1]]
```

```
[[1]]
[[1]][[1]]
[1] 1

[[1]][[2]]
[1]  TRUE FALSE FALSE
```

```r
# extract list level 3
list1[[1]][[1]]
```

```
[[1]]
[1] 1

[[2]]
[1]  TRUE FALSE FALSE
```

```r
# extract 1st member from list level 3
list1[[1]][[1]][[1]]
```

```
[1] 1
```

```r
# extract 2nd member from list level 3
list1[[1]][[1]][[2]]
```

```
[1]  TRUE FALSE FALSE
```

17 Matrices

A matrix is a two-dimensional structure that contains elements of the same type (numeric, character, or logical).

```
# Creating a 3x3 numeric matrix
my_matrix <- matrix(1:9, nrow = 3, ncol = 3)

# Accessing elements
my_matrix[1, 2]  # Access the element in row 1, column 2
```

```
[1] 4
```

```
# using matrix()
M <- matrix(data = 1:9, nrow = 3, ncol = 3)
M
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
M <- matrix(data = 1:9, nrow = 3, ncol = 3, byrow = T)
M
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
matrix(data = 1:6, nrow = 2, ncol = 3)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```r
# by merging multiple vectors
v1 <- c(1,2,3)
v2 <- c(4,5,6)
v3 <- c(7,8,9)

rbind(v1,v2,v3)
```

```
   [,1] [,2] [,3]
v1    1    2    3
v2    4    5    6
v3    7    8    9
```

```r
cbind(v1,v2,v3)
```

```
     v1 v2 v3
[1,]  1  4  7
[2,]  2  5  8
[3,]  3  6  9
```

```r
# by altering vector dimension
v <- 1:9
v
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```r
dim(v) <- c(3,3)
v
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```r
# Matrix properties

# rownames & colnames
M <- matrix(1:12, nrow = 4, dimnames = list(c("r1","r2","r3", "r4"), c("c1","c2","c3")))
M
```

```
   c1 c2 c3
r1  1  5  9
r2  2  6 10
r3  3  7 11
r4  4  8 12
```

```r
rownames(M)
```

```
[1] "r1" "r2" "r3" "r4"
```

```r
colnames(M)
```

```
[1] "c1" "c2" "c3"
```

```r
# matrix dimension
dim(M)
```

```
[1] 4 3
```

```r
# get all attributes
attributes(M)
```

```
$dim
[1] 4 3

$dimnames
$dimnames[[1]]
[1] "r1" "r2" "r3" "r4"

$dimnames[[2]]
[1] "c1" "c2" "c3"
```

```r
# change rownames & colnames
rownames(M) <- paste("row ", 1:4, sep = "")
colnames(M) <- paste("col ", 1:3, sep = "")
attributes(M)
```

```
$dim
[1] 4 3

$dimnames
$dimnames[[1]]
[1] "row 1" "row 2" "row 3" "row 4"

$dimnames[[2]]
[1] "col 1" "col 2" "col 3"
```

```
M
```

```
      col 1 col 2 col 3
row 1     1     5     9
row 2     2     6    10
row 3     3     7    11
row 4     4     8    12
```

```
# class and type
class(M)
```

```
[1] "matrix" "array"
```

```
typeof(M)
```

```
[1] "integer"
```

```
# check for matrix
is.matrix(M)
```

```
[1] TRUE
```

```
# Access matrix elements

# integer vector as index
M
```

```
      col 1 col 2 col 3
row 1     1     5     9
row 2     2     6    10
row 3     3     7    11
row 4     4     8    12
```

```
M[2,3]
```

```
[1] 10
```

```
M[c(1,2),3]
```

```
row 1 row 2
    9    10
```

```
M[c(2,3),] # selected rows and all columns
```

```
      col 1 col 2 col 3
row 2     2     6    10
row 3     3     7    11
```

```
M[,c(2,3)] # selected columns and all rows
```

```
      col 2 col 3
row 1     5     9
row 2     6    10
row 3     7    11
row 4     8    12
```

```
# logical vector as index
M[c(T,T,F,F), c(T,T,T)]
```

```
      col 1 col 2 col 3
row 1     1     5     9
row 2     2     6    10
```

```r
# character vector as index
M[c("row 2", "row 3"), c("col 1", "col 2")]
```

```
      col 1 col 2
row 2     2     6
row 3     3     7
```

```r
# range of indexes (slicing rows and columns)
M[1:3,2:3]
```

```
      col 2 col 3
row 1     5     9
row 2     6    10
row 3     7    11
```

```r
# Access matrix elements

# modify 1 element
M
```

```
      col 1 col 2 col 3
row 1     1     5     9
row 2     2     6    10
row 3     3     7    11
row 4     4     8    12
```

```r
M[1,1] <- 10
M
```

```
      col 1 col 2 col 3
row 1    10     5     9
row 2     2     6    10
row 3     3     7    11
row 4     4     8    12
```

```r
# modify more than one element
M[2:3,3] <- 20
M
```

```
      col 1 col 2 col 3
row 1    10     5     9
row 2     2     6    20
row 3     3     7    20
row 4     4     8    12
```

```
# modify elements based on condition
M[M>10] <- 0
M
```

```
      col 1 col 2 col 3
row 1    10     5     9
row 2     2     6     0
row 3     3     7     0
row 4     4     8     0
```

```
# transpose a matrix
t(M)
```

```
      row 1 row 2 row 3 row 4
col 1    10     2     3     4
col 2     5     6     7     8
col 3     9     0     0     0
```

```
# add row to matrix
M
```

```
      col 1 col 2 col 3
row 1    10     5     9
row 2     2     6     0
row 3     3     7     0
row 4     4     8     0
```

```
rbind(M, c(0,0,0))
```

```
      col 1 col 2 col 3
row 1    10     5     9
row 2     2     6     0
row 3     3     7     0
row 4     4     8     0
          0     0     0
```

```r
# add column to matrix
cbind(M, c(0,0,0, 0))
```

```
      col 1 col 2 col 3
row 1    10     5     9 0
row 2     2     6     0 0
row 3     3     7     0 0
row 4     4     8     0 0
```

```r
# alter matrix dimensions
dim(M)
```

```
[1] 4 3
```

```r
dim(M) <- c(3,4) # names are dropped
M
```

```
     [,1] [,2] [,3] [,4]
[1,]   10    4    7    0
[2,]    2    5    8    0
[3,]    3    6    9    0
```

```r
# merge 2 matrices
M1 <- matrix(data = rep(0,4), nrow = 2, ncol = 2)
M2 <- matrix(data = rep(1,4), nrow = 2, ncol = 2)
M1
```

```
     [,1] [,2]
[1,]    0    0
[2,]    0    0
```

```r
M2
```

```
     [,1] [,2]
[1,]    1    1
[2,]    1    1
```

```
rbind(M1,M2)
```

```
     [,1] [,2]
[1,]    0    0
[2,]    0    0
[3,]    1    1
[4,]    1    1
```

```
cbind(M1,M2)
```

```
     [,1] [,2] [,3] [,4]
[1,]    0    0    1    1
[2,]    0    0    1    1
```

```
#  Matrix arithmetics

# matrix  - scalar (scalar with each vector element)
M
```

```
     [,1] [,2] [,3] [,4]
[1,]   10    4    7    0
[2,]    2    5    8    0
[3,]    3    6    9    0
```

```
a <- 10

# Addition +
M + a
```

```
     [,1] [,2] [,3] [,4]
[1,]   20   14   17   10
[2,]   12   15   18   10
[3,]   13   16   19   10
```

```
# Subtraction -
M - a
```

```
     [,1] [,2] [,3] [,4]
[1,]    0   -6   -3  -10
[2,]   -8   -5   -2  -10
[3,]   -7   -4   -1  -10
```

```
# Multiplication *
M * a
```

```
     [,1] [,2] [,3] [,4]
[1,]  100   40   70    0
[2,]   20   50   80    0
[3,]   30   60   90    0
```

```
# Division /
M / a
```

```
     [,1] [,2] [,3] [,4]
[1,]  1.0  0.4  0.7    0
[2,]  0.2  0.5  0.8    0
[3,]  0.3  0.6  0.9    0
```

```
# Exponent ^  **
M^a
```

```
           [,1]     [,2]       [,3] [,4]
[1,] 1.0000e+10  1048576  282475249    0
[2,] 1.0240e+03  9765625 1073741824    0
[3,] 5.9049e+04 60466176 3486784401    0
```

```
# Modulus (Remainder from division) %%
M %% 2
```

```
     [,1] [,2] [,3] [,4]
[1,]    0    0    1    0
[2,]    0    1    0    0
[3,]    1    0    1    0
```

```
# Integer Division %/%
M %/% 2
```

```
     [,1] [,2] [,3] [,4]
[1,]    5    2    3    0
[2,]    1    2    4    0
[3,]    1    3    4    0
```

```r
# Other functions on matrix elements
sqrt(M)
```

```
         [,1]     [,2]     [,3] [,4]
[1,] 3.162278 2.000000 2.645751    0
[2,] 1.414214 2.236068 2.828427    0
[3,] 1.732051 2.449490 3.000000    0
```

```r
log(M)
```

```
          [,1]     [,2]     [,3] [,4]
[1,] 2.3025851 1.386294 1.945910 -Inf
[2,] 0.6931472 1.609438 2.079442 -Inf
[3,] 1.0986123 1.791759 2.197225 -Inf
```

```r
sum(M)
```

```
[1] 54
```

```r
# matrix - vector (matrix element to element | member-by-member)
M1 <- matrix(data = 1:9, nrow = 3, byrow = T)
M2 <- matrix(data = rep(3,9), nrow = 3)

# Addition +
M1 + M2
```

```
     [,1] [,2] [,3]
[1,]    4    5    6
[2,]    7    8    9
[3,]   10   11   12
```

```r
# Subtraction -
M1 - M2
```

```
     [,1] [,2] [,3]
[1,]   -2   -1    0
[2,]    1    2    3
[3,]    4    5    6
```

```r
# Multiplication *
M1 * M2
```

```
     [,1] [,2] [,3]
[1,]    3    6    9
[2,]   12   15   18
[3,]   21   24   27
```

```r
# Division /
M1 / M2
```

```
          [,1]      [,2] [,3]
[1,] 0.3333333 0.6666667    1
[2,] 1.3333333 1.6666667    2
[3,] 2.3333333 2.6666667    3
```

```r
# Exponent ^  **
M1^M2
```

```
     [,1] [,2] [,3]
[1,]    1    8   27
[2,]   64  125  216
[3,]  343  512  729
```

```r
# Modulus (Remainder from division) %%
M1 %% M2
```

```
     [,1] [,2] [,3]
[1,]    1    2    0
[2,]    1    2    0
[3,]    1    2    0
```

```r
# Integer Division %/%
M1 %/% M2
```

```
     [,1] [,2] [,3]
[1,]    0    0    1
[2,]    1    1    2
[3,]    2    2    3
```

```r
# matrix-matrix style multiplication
M1
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```r
M2
```

```
     [,1] [,2] [,3]
[1,]    3    3    3
[2,]    3    3    3
[3,]    3    3    3
```

```r
t(M1) %*% M2
```

```
     [,1] [,2] [,3]
[1,]   36   36   36
[2,]   45   45   45
[3,]   54   54   54
```

```r
M1 %*% M2
```

```
     [,1] [,2] [,3]
[1,]   18   18   18
[2,]   45   45   45
[3,]   72   72   72
```

```r
# matrix algebra (matrix based functions)
M <- matrix(data = c(1,5,3,2,4,7,4,6,2), nrow = 3, byrow = T)

# get diagonal elements
diag(M)
```

```
[1] 1 4 2
```

```r
# get matrix determinant
det(M)
```

```
[1] 74
```

```r
# get inverse of a matrix M^(-1)
solve(M)
```

```
            [,1]       [,2]        [,3]
[1,] -0.45945946  0.1081081  0.31081081
[2,]  0.32432432 -0.1351351 -0.01351351
[3,] -0.05405405  0.1891892 -0.08108108
```

```r
# get eigen values
eigen(M)
```

```
eigen() decomposition
$values
[1] 11.778446+0.0000000i -2.389223+0.7578106i -2.389223-0.7578106i

$vectors
             [,1]                  [,2]                  [,3]
[1,] 0.4687233+0i  0.5211486+0.2411697i  0.5211486-0.2411697i
[2,] 0.6544420+0i -0.6642393+0.0000000i -0.6642393+0.0000000i
[3,] 0.5932993+0i  0.4573822-0.1408153i  0.4573822+0.1408153i
```

```r
# calculate sum over rows or columns
M
```

```
     [,1] [,2] [,3]
[1,]    1    5    3
[2,]    2    4    7
[3,]    4    6    2
```

```r
rowSums(M)
```

```
[1]  9 13 12
```

```r
colSums(M)
```

```
[1]  7 15 12
```

```r
# Lets solve simple matrix equation
# A * X = B
A <- matrix(data = c(1,2,4,5), nrow = 2, byrow = T)
B <- matrix(data = c(5,24,17,66), nrow = 2, byrow = T)
# X = A^(-1) * B
X <- solve(A) %*% B
X
```

```
     [,1] [,2]
[1,]    3    4
[2,]    1   10
```

```r
# test
A %*% X # should get B
```

```
     [,1] [,2]
[1,]    5   24
[2,]   17   66
```

```r
# summarizing a matrix (apply)
M
```

```
     [,1] [,2] [,3]
[1,]    1    5    3
[2,]    2    4    7
[3,]    4    6    2
```

```r
# sum of rows
apply(X = M, MARGIN = 1, FUN = sum)
```

```
[1]  9 13 12
```

```r
# sum of columns
apply(X = M, MARGIN = 2, FUN = sum)
```

```
[1]  7 15 12
```

```r
# create matrix of random numbers
rnorm(n = 1000, mean = 0, sd = 2)
```

```
   [1]  2.3038665133 -0.5228105384 -1.0275129890  0.2665742075 -2.0391936244
   [6]  5.1593306678 -0.3507330551  0.3083353614  2.3497279742 -0.9668248187
  [11]  2.9223750939 -2.6592028254 -1.7454977355  1.0091531047 -1.7754530722
  [16] -1.2314826832 -0.4015295664  1.5082964100  1.9684154806 -1.1596955372
  [21]  0.1673862978  1.1466402125 -2.8462073629  1.5103405910 -0.5370264886
  [26] -1.0399799847 -3.6735816653 -0.8094433567 -0.8291799157 -0.7390376431
  [31] -0.6797445799 -0.8469492024  4.4005287367  1.7417981397  0.2663592446
  [36]  1.5265940412  1.9494638975 -0.3232114125 -0.5263972908  1.0628608415
  [41] -1.6875877821 -1.1342006308 -4.7536145181  1.7308837796  2.0589366271
  [46] -1.8091819044  1.6042822381  1.6924630248  0.8352609821  3.0132257402
  [51] -3.8672473656  2.4291190355  3.3766097595 -4.3778560674  1.0390371758
  [56]  0.3570569974 -4.2854199703  3.3640579104  1.4798360622 -0.3380219866
  [61] -4.0202400638  0.1996069208 -2.5652713177 -0.3770570974  0.8026996366
  [66] -0.5581805395 -0.6220977335 -1.4778988337 -2.3607956773 -0.1607682932
  [71]  0.4134713646  2.8973223467 -1.2412296519  1.3037767985 -0.4227890521
  [76] -0.1095143158 -2.2727950551  0.3456234015 -0.4283627342  0.9596445748
  [81]  3.7565881774 -0.0825824661 -2.0246519659  0.1362311165 -1.7708704600
  [86] -2.4064587971 -4.1091352895 -4.1543903723 -0.0097692582 -2.0882867374
  [91]  2.6834857309 -1.5062197445  1.1896178847  0.6170517221  3.2882826245
  [96]  1.3462927282  0.3552080155 -0.7919079503 -1.6737353576  1.9502233308
 [101] -1.1712609981 -1.4154092580 -0.2139666366 -1.5624678288  0.5017405625
 [106]  1.2857042309 -0.2768378562 -2.2367993228 -0.5681450797 -2.6473175490
 [111] -1.3933334471 -1.4045781522 -0.6289329284  0.5394777027  1.3613680274
 [116] -0.7606162301 -2.3500396075 -1.9596954469  0.9850453283 -0.0624958231
 [121]  0.8982655092  0.2325017539 -3.5555016495 -1.5048649877 -0.8167346186
 [126]  0.6878174046  1.6985320541  0.7623894717 -0.7541933634 -0.3041748666
 [131] -1.4989573816  1.1522537379 -0.9889288745  1.4748020735  2.3634189552
 [136]  0.6503768893 -2.7995527672 -2.4836763110  1.5964448787 -0.3003866858
 [141]  1.0779142118  2.6063305936  2.0821976990  3.0058954670 -2.6719878300
 [146] -2.6996042117  0.9959985883  0.4799341658  2.5616864378 -0.4712361310
 [151] -1.0858091146 -2.1677326880 -0.8875011488  0.5322569878 -3.2690102151
 [156]  2.0518227542 -0.3104068929  1.8478619377  3.8619048582 -2.0597329429
 [161] -1.3469641808 -0.4742414615  2.4024790574  0.8616031424 -1.2765538879
 [166]  0.7042941373  1.8330626247 -0.2015265980 -2.6748620505 -0.3361644807
 [171] -0.5661188912  0.2758229591 -1.7089604320 -0.7467837517  0.8178739907
 [176]  0.9419106281 -0.4615134045  2.7093570534 -1.0521094758 -2.6152117972
 [181] -2.0029596518 -0.8818069884 -0.5563323605 -0.0202192638 -1.1844602869
```

```
[186] -0.7002193916 -0.6100073645 -1.1310965088 -0.1412911358  3.1924313676
[191] -0.8001445400 -1.1781638752 -0.0947552958  1.9260168146 -0.5045728791
[196] -2.0607349289  0.1299871470  1.1179030868 -0.4917118141 -1.0112001883
[201]  4.6477138059 -3.6382846103 -1.2471039173  1.4117956916  2.6330534949
[206]  2.4914580940 -1.1244166477  2.6027404437  1.4422522992  1.7160215815
[211] -1.2351168221  1.6041210465  0.2017559047 -0.0807318755  2.1843481377
[216] -2.1183465382 -0.7587406959  1.0957011200  2.2528985173  1.9272020355
[221]  0.3913611165  1.1364951372  2.4266418982 -3.0009488604  1.0408213318
[226] -1.3151039999 -0.9846103933 -2.8996024903 -2.3613732139 -1.7131710748
[231] -4.0112202492  4.4653702696  2.4276381843 -2.2710607856  0.0188407511
[236]  2.1273565468 -1.0450939547  0.8094146807 -1.0125393088  0.5887050826
[241] -1.0663603214 -0.2829944630  0.4652761594  1.6659433216  2.9492818248
[246]  1.9558534203  1.0474324012  2.5133208160 -0.2053775960 -2.7321498709
[251]  1.1334921535  0.1790859918 -0.3842880130 -0.3720047805 -1.1489126119
[256]  3.1600229626 -1.9685357447 -0.5667255897 -5.8787231904  2.4986517665
[261] -2.6179651359 -0.9342268979 -0.8511527508  2.4541753466  0.3749948005
[266] -1.5102347723  0.5916257154  1.6174567149 -0.5398542925 -0.1263869744
[271] -0.2658299138  3.3934488842  0.6549090146  0.9897630536  2.4770371249
[276] -1.5880555169  0.3756625627 -0.6492497715  1.5991089490  0.7251758760
[281] -0.1269571119  1.2759973726 -0.1626150131  0.1666286696 -2.2952724947
[286]  1.6927771407 -0.1416028415 -1.8988386746  0.7620898421  0.3941171780
[291]  3.9996387374 -2.7159424152 -1.1969473072  0.4348451802 -1.0646323395
[296]  2.5967053492  1.1925040565  2.3479995369 -0.7969631169 -3.2992519390
[301]  2.6856813262 -1.1715417496 -1.3938383445  0.5719085095 -2.6938939084
[306] -0.9029590630 -4.3970918279 -1.0226513378 -2.7180776079  2.0767558519
[311] -0.8559842884  2.5165526873 -1.6364465862 -1.4010520016 -1.8894253212
[316] -1.9465125738 -1.4043413831 -1.2340256238  3.3888073462  0.0394651277
[321] -0.1019021645  0.5376874322 -1.2620510587 -2.1420162071  1.6047550315
[326] -0.8180541317  2.9881324475  2.2111078446  0.7523305422  0.5895079136
[331] -2.8860996012 -0.7683353485 -2.0514700400 -3.9944776522 -3.1700533882
[336] -1.8676403788  1.1047150563 -0.1779376634 -2.5357204674  0.8847764505
[341]  0.2864987250  2.3871057451  0.7694696826  0.0600636942 -2.9717186663
[346]  0.3833814320 -0.5912675940 -2.2839116904 -0.9113666677 -0.0006553572
[351]  1.0887168765 -4.3950852431 -0.1444507287 -3.1778376219  1.0644019571
[356] -0.5230813071 -4.0270536455 -3.0370250943 -1.2351072246 -0.4561899091
[361] -0.7437277886  3.8381715874  1.8487318449 -0.4569584826  2.1067232847
[366] -3.7414424870 -1.5059469662 -0.1681085367  1.0217226878 -4.2252101665
[371] -1.4888316388  0.9751848176 -2.1735453196 -0.5625383348  0.5406634027
[376] -4.0651742386  2.5160310645  4.9136628518 -0.3512909272 -2.1409239454
[381]  0.0869604080  0.8217489432  0.4525570154 -0.0833427159 -0.0339226718
[386]  4.6211015982 -0.5580465945 -3.5016641318 -2.8147681431 -0.1059019851
[391] -0.3995441388  1.4104984408 -1.5138918931 -1.5086935793  1.6720792410
[396] -3.5667448763  0.9039954858 -2.2748873342  0.9232234542  0.4634914709
```

```
[401]  0.9412188440  1.4584212670  0.7423723663 -1.0353635665 -1.4223246706
[406]  1.0695719145  1.4126475824 -1.4197524541  0.0205289850 -0.4384723870
[411] -2.4209298863 -5.5534506213  1.1226846220  1.1484648283 -0.4639787177
[416] -4.1813691626  2.0396438313  0.7105997701 -0.4668955253  1.2857564981
[421] -4.0824372544  1.1737052945  3.6376537205  2.5465070930  0.1848911984
[426] -2.1354015325  4.2737743608 -0.8695698449  3.3816163037 -2.5864088592
[431]  0.4570500268  0.8927100206 -1.8268908482 -2.2560880212 -0.7527119363
[436]  1.4130103409 -0.0816446180 -0.3258005050  0.9954251429 -3.0940376166
[441] -4.4546201977 -3.4244791179  1.9310683361  2.9557773520 -1.4414923838
[446]  1.8248404251  1.5719828072 -2.4600165076 -0.9848391602 -0.4989512255
[451] -0.1166952206 -0.7497946541  1.1140931421 -0.1778938946  1.5926309130
[456] -0.0905290986  0.6954067891 -0.5255996639  1.0547070144  4.8450869398
[461]  2.4110267297  2.9134768704 -0.6522256274  2.6167435472 -1.3285598546
[466]  0.1686427484  3.7549733141  0.8650140480 -1.0557132036 -1.4217091697
[471]  2.3759246598 -1.6842356054  4.2648210803  1.0638366887  1.2150754052
[476] -1.1222284050 -0.4097836649 -3.6432829146 -1.1008544417  1.1169994637
[481] -0.3962144958 -3.1354836530  0.2929873716  1.1337440683  3.0660026074
[486]  0.1859391011  2.2005309521 -2.4267519884 -0.9252283584  1.1975880600
[491] -0.1538192042  3.1854702319 -3.1640866205  2.6019042774  1.9225162744
[496]  1.1439842091  2.5844611096  0.8806912446 -0.2477399713 -0.9231850013
[501] -1.7460187940  1.0736097966  1.6092961129 -0.4509569343 -4.7744849116
[506] -3.0201610931  2.7894250576  0.8857412677  1.0369547310  0.5027550730
[511]  2.5034575912 -0.3209930524  1.9866178601 -0.5224893834  3.0129406748
[516] -3.3797941847 -2.0173935781  0.0821395570 -0.9218658537 -1.7160261009
[521] -3.4868012289  0.7458469902  1.1884224642 -1.4076617867  0.4824828829
[526] -0.7703267316  0.8380453872  0.7775231926 -0.4960500578 -1.4578268010
[531] -1.9124402339 -2.1231609762  0.2972104402  0.9707814229 -1.9587946775
[536]  2.3965818941 -1.7137360389 -1.7308558237  1.7605145539 -2.6504231769
[541] -0.4881242694  1.5641972595  1.3227463328 -2.1233073271  3.5289106982
[546]  1.3406030191  1.8412676258  2.1077477104 -0.9111516031  0.4216398883
[551]  1.4994834686 -0.3081056740 -3.2203397348  0.9931171117  0.7340096539
[556] -2.0391196885  0.4767557199  0.9544379878  2.0789759560  1.7625148444
[561] -0.6663547470  1.5890910500 -0.8016222443 -0.2705646424 -1.0138880178
[566] -2.1981842274 -1.7149310978 -0.7913104746  0.9714652382 -0.2497921551
[571] -0.4020684965 -3.2533940464  0.3616937275  0.4509861353 -0.1554752413
[576]  1.0974473880 -0.2134572796 -1.8187083556 -1.8316647219  0.9681173906
[581]  3.3455251882  1.3142264160  0.2331592221  3.1010853239 -0.9008966600
[586] -3.0435990099  2.1590167829  2.4747474411 -0.5627655465 -1.7256250906
[591]  0.6899606385 -1.1434601111  0.0693180602  1.1057694957 -1.0106127100
[596]  1.6877324928 -0.3116986888  2.5040861617  2.4275769327 -2.4252127903
[601] -2.0214723605 -3.2049568180 -2.8889253167 -0.1144807770 -1.4376211802
[606] -2.8504605637  0.2184734739 -1.3884431893  0.9146784223  1.4966821320
[611]  2.3784887992 -0.8417928857 -3.3324684622 -1.1219401383  0.8818777890
```

```
[616] -2.0978323251 -1.3004073745  1.3277614180 -0.0426629988  2.7250109114
[621]  0.1701267525  4.8220866263  4.4890303785  2.0526693702 -0.0218566962
[626] -0.5108018337 -0.3072531964 -0.5311544790  0.5855962839 -1.4614738672
[631] -0.1343838853 -1.0108516214 -0.3237256518 -1.1729953363 -0.9068796405
[636]  1.7913622191 -0.4371528366 -0.0699158139  0.6524355744 -0.8974395677
[641]  1.4389594342 -1.4034114013 -0.6156677479 -1.1152395452  1.4965636897
[646] -1.7372989109 -0.0876249005  3.6062319108  1.1590371842 -0.1960442372
[651]  1.6741969684 -1.8772522380 -0.8797288404 -0.7260877022 -1.7835284326
[656] -0.3231138683 -2.2705531360 -2.3801212537 -0.9090031424  0.9806245121
[661] -1.0359212279 -4.7119740095  3.2978703609 -0.7991363675 -2.7720887953
[666] -0.6566168639  2.4932188674  2.5559850255  1.0100431161 -3.8141420006
[671] -2.0298677626 -0.9305558998  0.3424514719  1.6815342520 -3.6563096971
[676]  1.1634274336 -3.7787112385  1.4031908719  0.9081650939  0.6983921031
[681] -0.6465264412  4.9134199497 -1.4463789527 -0.6977673045 -0.6474654441
[686] -0.7987085563 -0.9406170343 -3.5971699584 -0.1084678896 -0.0617437012
[691] -0.8674068899  1.6384520294  0.0138657693 -2.8501641620  1.4065058559
[696]  0.3497355342  1.3460832451 -0.8494822222 -1.7872989798  1.0539115224
[701] -1.7955179622  1.4499730517 -0.7019574254 -4.5069135105  3.5271901528
[706] -0.9610760908 -2.9072189211 -1.2042150586  1.8090061779  2.5448723853
[711] -0.1739022529  3.0622941382 -2.3080445818 -2.2159860008 -1.8912110918
[716]  2.8506464581  1.7592833291  3.4609407866 -1.9753867596  0.4139945608
[721]  1.7631326439  0.6454854119 -3.1260742294 -1.9362819734  3.2953302006
[726]  1.5567284996  3.4396332204 -1.5934386819  0.9511903693 -0.1870834332
[731] -2.8261626962  1.9990693956 -4.9112595669 -0.0071740456 -1.1840237758
[736]  1.4510162349  1.1759549428 -4.0773310164  0.2931199702  1.3043723404
[741]  2.8257211478 -0.2035650753  1.6050312814  2.7263820198  0.0258043760
[746]  2.8596666483 -1.3790151890 -0.4160475190  7.0185711470  2.8865286843
[751]  3.4254598535 -1.5565753642 -2.2634713953 -2.3580888611  0.5034863553
[756]  0.2026637286 -1.2949897413  3.1174796624  0.1914081942 -3.8238544863
[761]  1.8053451560  0.1194689325 -7.2341713704  0.9779245575  1.3215936844
[766]  0.8058171258  1.3426870313 -2.5539992390 -0.8694181795 -0.9765778660
[771] -0.0174019544 -0.5431264235 -0.5274268713 -2.3839052412 -1.8258964101
[776]  1.7792225024  0.6549659448 -0.2314683811  3.4339437981  6.6009998743
[781]  3.6064434751  5.7739440266 -0.9872286584  2.2579782216  1.7029405671
[786]  0.1308858273 -2.0909094754 -1.8635864800  2.2869799980 -0.5659175882
[791] -0.9387090668 -2.3403080138  0.9965504179  1.0060158176  3.6238433334
[796]  0.3890900384  0.4579384148 -1.6111293230 -0.5118905107  2.2153973085
[801] -3.0349221715 -2.4652188778 -0.5336040239  0.2054786255  5.0819996828
[806] -2.1236573188 -0.0134871400  2.0479350556  3.6630916891  1.5751362293
[811] -1.8264729436  2.4893849257 -0.0905505498  2.6461369428 -1.9635757382
[816]  1.1065341686 -1.0324519451 -0.2654420054  1.0756574775 -2.3706571752
[821] -2.5182554640  3.0055465824  2.1671873534  2.1137357039 -1.7225724098
[826] -1.5325202882  0.6555622816  0.9692241624 -3.3206848790  0.1527211780
```

```
 [831]  0.0594987692  0.5370905544  0.2013760587  3.7741703652  2.8516365712
 [836] -2.6495723239 -0.0349663290  0.0445957057 -0.6550029546 -3.7822178221
 [841]  1.9383184486  3.6090750003  0.5324145598 -1.0858702482 -0.0277234081
 [846] -0.3511246861 -1.7783497570 -0.6712750492 -2.2702616147 -3.4910070309
 [851] -2.7369444535 -3.0812251471 -1.5660700963  0.6222287184  0.5212806744
 [856] -2.0109099893  2.1067848434 -0.9849854114 -0.6865842338 -1.9707941607
 [861] -1.4785411897 -4.2449514869  0.0797088768 -0.7783429177 -2.3510730347
 [866] -0.5432461541 -2.3203525616 -0.5874995866  2.4733560701  3.8751331371
 [871]  0.6247986992  2.4356001964  2.0681927925  1.5050547979 -0.5125832206
 [876]  0.3137533885 -0.1164174146  0.1399700856  0.7924064754  0.8841788101
 [881]  0.3442850268 -0.4980370588 -0.6046278982  0.5089661676 -3.3650424025
 [886]  3.1596731250  1.1795426037  0.8496695519 -1.7758240983  2.5309840747
 [891]  0.4971643012  2.4448474535 -1.3068366481 -0.4856617953 -0.3824557655
 [896] -0.7866447238  3.0116406113  5.2287803622  2.8701882005  0.4938318669
 [901]  1.8493022135 -0.4434492632 -0.1312722343  1.8480478168 -0.4972822564
 [906]  1.6310744318  0.6748562196 -2.8285891357 -4.2346309750 -1.1934093488
 [911] -3.0982716848 -1.2037160180  4.1066965781 -1.2027504270  0.0025007701
 [916] -2.3216744780 -1.7378922514 -2.1449732431  1.4982736917 -2.3299984003
 [921]  1.4948303226 -0.4163438765 -0.3600332046 -1.2729168464 -1.1132524045
 [926] -3.8270418789  0.7913145425 -1.4482578953 -1.8376555205 -3.1149243109
 [931] -1.6343814965  1.2911001313  1.0778490957 -0.8564478509  4.9541342741
 [936]  0.0777951825 -2.3904118397  0.8479890542  2.3949253382 -2.5746555090
 [941] -2.1973338886  1.6413642925 -1.7599792761  0.4484151860 -0.0598899387
 [946]  3.4412544695  4.6976415211 -0.2738999407  0.7943736596 -0.4977593410
 [951] -0.1813151036 -3.2812801349  0.8346261820 -0.7914847379  3.0988099174
 [956]  1.8671756324 -0.9727410554 -0.9389609555 -2.1588695917 -3.5196655903
 [961] -0.9356099595 -0.3012233887  1.0560362753  1.4837952139  2.9471346535
 [966]  0.1249932020 -0.7381294103 -1.4597548903 -1.2502222097 -2.0713690088
 [971]  5.3726001501  1.0011869493 -1.4382582341  0.7925141707  2.5995623853
 [976]  1.3314177694  0.3838014271  3.9613340184 -2.7448780650  2.9119000132
 [981]  2.6796234417  0.5722142888 -0.6656106180  1.8780775094  0.8559491888
 [986]  2.0835563051  3.0098209058  1.1808787568 -1.3095394337 -2.6175117578
 [991]  2.1429348939 -2.2108760677 -4.5407290521  4.0764908331 -2.4597663052
 [996] -0.7391657255 -0.7776726057 -1.9067784390 -1.1734449084 -1.8436271369
```

```r
A <- matrix(data = rnorm(n = 1000, mean = 0, sd = 2), nrow = 100, ncol = 10)

# get mean over columns
apply(A, 2, mean)
```

```
 [1]  0.37524432 -0.16650864  0.09238770 -0.15095026  0.01241378 -0.22562517
 [7]  0.33851783 -0.26560568  0.07463225 -0.03932246
```

```r
# get mean over rows
apply(A, 1, mean)
```

```
  [1]  0.76206135 -0.40209295 -0.03097052 -0.58347007  0.29970241  0.01134068
  [7]  0.40338628  0.39931481  0.40827443  0.64523599  0.07031438 -0.18576137
 [13] -0.30980799 -0.12915700  0.50602682 -0.96888232 -1.18475725 -0.34818237
 [19]  0.01282864  0.25775176  0.05744428 -0.04751725 -0.08883312  0.48650232
 [25]  0.05451674 -0.33506994 -0.50903709  0.72466933  0.47875423 -0.25444715
 [31]  0.65158228 -0.18920777  0.05239992  1.00691657 -0.05246734  0.31105427
 [37] -0.21954818  0.29705510  0.30330812  0.49824560 -0.76964482 -1.10144149
 [43] -0.63821188  0.29401481 -0.05289494  0.10571283 -0.44664723  0.33007136
 [49] -1.20080946 -0.01604752 -0.28141624  1.09790790  1.11155381  0.42211178
 [55]  0.14955085  0.52398057 -0.28600644  0.39531524  0.08374000  0.76823831
 [61]  0.38478451  0.31547155 -0.11573042  0.93715687  0.04691260  0.66870138
 [67] -0.15122545  0.07586272 -1.06346099  0.09513931 -0.19567177 -1.04696372
 [73] -0.99826747 -0.72761097  0.88045306  0.79812710 -0.69712817 -0.11850553
 [79]  0.05343208 -0.53478684  0.51216792  0.41016947 -0.04359805 -0.23873001
 [85] -1.02041135 -0.99483857  0.91001737  0.35984084  0.36789233 -0.32058612
 [91] -0.36117155 -0.32860655 -0.36413383  0.67776300 -0.04685696  0.40948139
 [97] -0.68917037 -0.62110052  0.09181500 -0.21335074
```

```r
# calculate standard deviation for each column
apply(A, 2, sd)
```

```
 [1] 1.864360 1.961886 1.979902 2.298694 2.017087 1.985955 1.929059 2.045148
 [9] 1.872654 1.927609
```

18 Data Frames

A data frame is a table where each column can contain elements of different types (e.g., numbers, strings). It's the most common structure used for data sets.

```
# Creating a data frame
my_data <- data.frame(
  Name = c("Alice", "Bob", "Charlie"),
  Age = c(23, 30, 25),
  Gender = c("F", "M", "M")
)

my_data
```

```
     Name Age Gender
1   Alice  23      F
2     Bob  30      M
3 Charlie  25      M
```

```
# Accessing columns
my_data$Name  # Output: "Alice", "Bob", "Charlie"
```

```
[1] "Alice"   "Bob"     "Charlie"
```

```
# create data frame
df1 <- data.frame(col1 = 1:3,
                  col2 = c("a", "b", "c"),
                  col3 = c(T, F, T),
                  col4 = c(as.Date("2020-01-01"), as.Date("2020-01-03"), as.Date("2020-01-03"

# create data frame - vectors
col1 <- seq(10,100,10)
col2 <- seq(as.Date("2020-01-01"), length = 10, by = "weeks")
col3 <- rep("word", 10)
```

```r
df2 <- data.frame(num = col1,
                  date = col2,
                  string = col3)

# check DF structure
str(df2)
```

```
'data.frame':   10 obs. of  3 variables:
 $ num   : num  10 20 30 40 50 60 70 80 90 100
 $ date  : Date, format: "2020-01-01" "2020-01-08" ...
 $ string: chr  "word" "word" "word" "word" ...
```

```r
# create data frame - matrix
M <- matrix(data = 1:100, nrow = 10, ncol = 10, byrow = T)
rownames(M) <- paste("row", 1:10, sep = "")
colnames(M) <- paste("col", 1:10, sep = "")
M
```

```
      col1 col2 col3 col4 col5 col6 col7 col8 col9 col10
row1     1    2    3    4    5    6    7    8    9    10
row2    11   12   13   14   15   16   17   18   19    20
row3    21   22   23   24   25   26   27   28   29    30
row4    31   32   33   34   35   36   37   38   39    40
row5    41   42   43   44   45   46   47   48   49    50
row6    51   52   53   54   55   56   57   58   59    60
row7    61   62   63   64   65   66   67   68   69    70
row8    71   72   73   74   75   76   77   78   79    80
row9    81   82   83   84   85   86   87   88   89    90
row10   91   92   93   94   95   96   97   98   99   100
```

```r
df3 <- as.data.frame(M)
df3
```

```
      col1 col2 col3 col4 col5 col6 col7 col8 col9 col10
row1     1    2    3    4    5    6    7    8    9    10
row2    11   12   13   14   15   16   17   18   19    20
row3    21   22   23   24   25   26   27   28   29    30
row4    31   32   33   34   35   36   37   38   39    40
row5    41   42   43   44   45   46   47   48   49    50
row6    51   52   53   54   55   56   57   58   59    60
```

```
row7    61   62   63   64   65   66   67   68   69    70
row8    71   72   73   74   75   76   77   78   79    80
row9    81   82   83   84   85   86   87   88   89    90
row10   91   92   93   94   95   96   97   98   99   100
```

```r
# check DF dimensions
dim(df3)
```

```
[1] 10 10
```

```r
nrow(df3)
```

```
[1] 10
```

```r
ncol(df3)
```

```
[1] 10
```

```r
# check DF type / class
class(df3)
```

```
[1] "data.frame"
```

```r
typeof(df3)
```

```
[1] "list"
```

```r
# Accessing DF

# let's create DF - employees
df_emp <- data.frame(id = 1:6,
                     name = c("Max", "Jane", "John", "Tony", "Janis", "Helen"),
                     surname = c("Gordon", "Smith", "Don", "Price", "Jett", "Dust"),
                     age = c(55, 35, 46, 22, 60, 27),
                     date_start_work = c(as.Date("1985-09-01"), as.Date("2010-10-01"), as.Da
                     gender = c("M", "F", "M", "M", "F", "M"),
                     manager_position = c(T, F, F, F, T, F)
                     )
```

```r
# extract data as data frame (one column) - []
df_extr <- df_emp["name"]
df_extr
```

```
   name
1   Max
2  Jane
3  John
4  Tony
5 Janis
6 Helen
```

```r
class(df_extr)
```

```
[1] "data.frame"
```

```r
# extract data as vector (one column) [[]] $
df_extr <- df_emp[["age"]]
df_extr
```

```
[1] 55 35 46 22 60 27
```

```r
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```r
df_extr <- df_emp$age
df_extr
```

```
[1] 55 35 46 22 60 27
```

```r
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```r
# extract multiple columns
df_extr <- df_emp[c("name", "age")]
df_extr
```

```
   name age
1   Max  55
2  Jane  35
3  John  46
4  Tony  22
5 Janis  60
6 Helen  27
```

```r
# data frame slicing
df_emp
```

```
  id  name surname age date_start_work gender manager_position
1  1   Max  Gordon  55      1985-09-01      M             TRUE
2  2  Jane   Smith  35      2010-10-01      F            FALSE
3  3  John     Don  46      1999-06-01      M            FALSE
4  4  Tony   Price  22      2019-03-01      M            FALSE
5  5 Janis    Jett  60      1980-04-15      F             TRUE
6  6 Helen    Dust  27      2015-02-20      M            FALSE
```

```r
#extract second row in name column (1 cell)
df_emp[2,2]
```

```
[1] "Jane"
```

```r
df_emp[2,"name"]
```

```
[1] "Jane"
```

```r
# extract first 4 rows of last 2 columns
df_emp[1:4, 6:7]
```

```
  gender manager_position
1      M             TRUE
2      F            FALSE
3      M            FALSE
4      M            FALSE
```

```r
df_emp[1:4, c("gender", "manager_position")]
```

```
  gender manager_position
1      M             TRUE
2      F            FALSE
3      M            FALSE
4      M            FALSE
```

```r
# extract first column (all rows)
df_emp[,1]
```

```
[1] 1 2 3 4 5 6
```

```r
df_emp[,"id"]
```

```
[1] 1 2 3 4 5 6
```

```r
df_emp$id
```

```
[1] 1 2 3 4 5 6
```

```r
# extract last 2 rows (all columns)
df_emp[5:6,]
```

```
  id  name surname age date_start_work gender manager_position
5  5 Janis    Jett  60      1980-04-15      F             TRUE
6  6 Helen    Dust  27      2015-02-20      M            FALSE
```

```r
cols <- colnames(df_emp)
df_emp[5:6, cols]
```

```
  id  name surname age date_start_work gender manager_position
5  5 Janis    Jett  60      1980-04-15      F             TRUE
6  6 Helen    Dust  27      2015-02-20      M            FALSE
```

```r
# Modifying data frame

# append column
df_emp <- cbind(df_emp, role = c("director", "secretary", "analyst", "researcher", "CEO", "ar
df_emp$new_col <- 1

# append rows
df_emp <- rbind(df_emp, list(7, "Mark", "Jax", 32, as.Date("2020-01-01"), "M", F, "researcher

# problem with factor variables (new values not in factor levels)
# easy solution - append new row as data frame (rbind 2 data frames)!!!
# will show few rows later

# remove column
df_emp$new_col <- NULL

# remove row
df_emp <- df_emp[-7,]

# merge two data frames (row wise)
df_new_emp <- data.frame(id = 7,
                         name = "Mark",
                         surname = "Jax",
                         age = 32,
                         date_start_work = as.Date("2020-01-01"),
                         gender = "M",
                         manager_position = F,
                         role = "researcher")

df_emp <- rbind(df_emp, df_new_emp)

# merge two data frames (column wise)
df_attr <- data.frame(eye_color = c("blue", "green", "brown", "hazel", "blue", "brown", "brow
                      hair_color = c("blonde", "light brown", "black", "brown", "blonde", "da
df_emp <- cbind(df_emp, df_attr)


# Tips

# Df summary
summary(df_emp)
```

```
       id          name              surname               age
 Min.   :1.0   Length:7           Length:7           Min.   :22.00
 1st Qu.:2.5   Class :character   Class :character   1st Qu.:29.50
 Median :4.0   Mode  :character   Mode  :character   Median :35.00
 Mean   :4.0                                         Mean   :39.57
 3rd Qu.:5.5                                         3rd Qu.:50.50
 Max.   :7.0                                         Max.   :60.00
 date_start_work         gender          manager_position     role
 Min.   :1980-04-15   Length:7           Mode :logical    Length:7
 1st Qu.:1992-07-16   Class :character   FALSE:5          Class :character
 Median :2010-10-01   Mode  :character   TRUE :2          Mode  :character
 Mean   :2004-05-06
 3rd Qu.:2017-02-24
 Max.   :2020-01-01
  eye_color          hair_color
 Length:7           Length:7
 Class :character   Class :character
 Mode  :character   Mode  :character
```

```
# rows subsetting
subset(x = df_emp, gender == "M")
```

```
  id  name surname age date_start_work gender manager_position       role
1  1   Max  Gordon  55      1985-09-01      M             TRUE   director
3  3  John     Don  46      1999-06-01      M            FALSE    analyst
4  4  Tony   Price  22      2019-03-01      M            FALSE researcher
6  6 Helen    Dust  27      2015-02-20      M            FALSE    analyst
7  7  Mark     Jax  32      2020-01-01      M            FALSE researcher
  eye_color hair_color
1      blue     blonde
3     brown      black
4     hazel      brown
6     brown dark brown
7     brown      brown
```

```
subset(x = df_emp, gender == "F" & manager_position == T)
```

```
  id  name surname age date_start_work gender manager_position role eye_color
```

```
5  5 Janis    Jett  60      1980-04-15      F             TRUE  CEO      blue
  hair_color
5     blonde
```

```
rows <- which(df_emp[,"gender"] == "M")
df_emp[rows,]
```

```
  id  name surname age date_start_work gender manager_position       role
1  1   Max  Gordon  55      1985-09-01      M             TRUE   director
3  3  John     Don  46      1999-06-01      M            FALSE    analyst
4  4  Tony   Price  22      2019-03-01      M            FALSE researcher
6  6 Helen    Dust  27      2015-02-20      M            FALSE    analyst
7  7  Mark     Jax  32      2020-01-01      M            FALSE researcher
  eye_color hair_color
1      blue     blonde
3     brown      black
4     hazel      brown
6     brown dark brown
7     brown      brown
```

```
rows <- which(df_emp[,"gender"] == "F" & df_emp[,"manager_position"] == T)
df_emp[rows,]
```

```
  id  name surname age date_start_work gender manager_position role eye_color
5  5 Janis    Jett  60      1980-04-15      F             TRUE  CEO      blue
  hair_color
5     blonde
```

```
# some calculations regarding data frames
nr_managers <- sum(df_emp$manager_position)
mean_age <- mean(df_emp$age)
df_emp$name_surname <- paste(df_emp$name, df_emp$surname, sep = " ") # merge name and surname

# use apply to sum over columns (age, manager_position)
apply(df_emp[,c("age", "manager_position")], 2, sum)
```

```
             age manager_position
             277                2
```

19 Factors

Factors are used to represent categorical data. They store both the data values and the corresponding levels.

```
gender_factor <- factor(c("Male", "Female", "Male"))

# Display the factor and its levels
print(gender_factor)
levels(gender_factor)


# create factor variable (gender)
gender <- factor(x = c("male", "female", "female"))

# check new variable
gender
str(gender)
class(gender)
typeof(gender)

# create with ordering
gender <- factor(x = c("male", "female", "female"), ordered = T)
is.ordered(gender)

# check levels
levels(gender) # order of levels based on variable (string alphabetic order)

# we can define our own levels (custom levels order)
gender <- factor(x = c("male", "female", "female"), levels = c("male", "female"), ordered = I
gender
levels(gender)

# factor properties
levels(gender)
is.factor(gender)
is.ordered(gender)
```

```r
# create other object to factor
strings <- c("a", "b", "a", "c")
f_strings <- factor(strings)

#f_string
```

20 Arrays

Arrays are similar to matrices but can have more than two dimensions.

```
# Creating a 3-dimensional array
my_array <- array(1:24, dim = c(3, 4, 2))

# Accessing elements
my_array[1, 2, 1]  # Access the element in the first dimension, second row, and first slice
```

```
[1] 4
```

21 Summary

- `Vector:` One-dimensional, homogeneous.
- `List:` One-dimensional, heterogeneous.
- `Matrix:` Two-dimensional, homogeneous.
- `Data Frame:` Two-dimensional, heterogeneous (columns can be different types).
- `Factor:` Categorical data representation.
- `Array:` Multi-dimensional, homogeneous.

22 Manipulating Vectors, Data Frames, and Lists

```
library(haven)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(tidyr)
```

23 Vectors: Indexing & Vectorized Ops

```
#Creating vector
x=1:10
x=10:1
x=-5:10
x=c(1:10)
x=c(-5:10)
x=c(1,2,3,4,5,6,7,8,9,10)

# Naming a vector
a=c(1:3)
names(a) #Returns null
```

```
NULL
```

```
names(a)=c("one","two","three")
names(a)
```

```
[1] "one"   "two"   "three"
```

```
a
```

```
  one   two three
    1     2     3
```

```
#2)Accessing vector element
x=c(1,3,5,7)
x[2]
```

```
[1] 3
```

```r
x[c(1,3)]
```

```
[1] 1 5
```

```r
x[-2]
```

```
[1] 1 5 7
```

```r
x[-c(1,3)]
```

```
[1] 3 7
```

```r
x[0]
```

```
numeric(0)
```

```r
y=x[10]
class(y)
```

```
[1] "numeric"
```

```r
#Note: X[0],x[10],output is numeric class only
#***********************************
#3)Modification of Vector elements
x=c(9,3,5,7)
x[2]=13
x[-c(2,3)]=c(11,17)
x[-1]=c(110,170,70)

#*****************************
x=c(11,3,5,7)
x[9]=x[7]
#*****************************
x=c(11,3,5,7)
x[9]=x[2]

x=c(1,3,5,7)
x[2]=x[11]
```

```
#***************************
x=c(1,3,5,7)
x[c(2,5)]=x[c(4,4)] # It assign 4 element of to 2nd element and 4th element to 5th element
#***************************           from x=1,3,5,7
x=c(11,3,5,7)
x[c(2,7)]=x[c(1,3)]

x=c(1,3,5,7)
x[c(2,3)]=x[c(1,10)]

#4)Airthematic Operations on Vector
x=c(1,3,5,7)
x+10
```

```
[1] 11 13 15 17
```

```
x-5
```

```
[1] -4 -2  0  2
```

```
x*10
```

```
[1] 10 30 50 70
```

```
x/10
```

```
[1] 0.1 0.3 0.5 0.7
```

```
x=c(1,3,5,7)
x%/%2
```

```
[1] 0 1 2 3
```

```
x=c(1,3,5,7)
x%%2
```

```
[1] 1 1 1 1
```

```r
min(x)
```

```
[1] 1
```

```r
max(x)
```

```
[1] 7
```

```r
median(x)
```

```
[1] 4
```

```r
mean(x)
```

```
[1] 4
```

```r
range(x)
```

```
[1] 1 7
```

```r
var(x)
```

```
[1] 6.666667
```

```r
sd(x)
```

```
[1] 2.581989
```

```r
quantile(x)
```

```
  0%  25%  50%  75% 100%
 1.0  2.5  4.0  5.5  7.0
```

```r
quantile(1:20,probs=c(.25,0.9))
```

```
  25%   90%
 5.75 18.10
```

```
IQR(x)
```

```
[1] 3
```

```
#5)"WHICH" function
x=c(2,3,4,5,11,112,133,33)
x>5
```

```
[1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE
```

```
x=c(2,3,4,5,11,112,133,33)
y=which(x>5) #Returns the position,not values
y=x[which(x>5)]
y=x[x>5]

x=c(2,3,4,5,11,112,133,33)
min(x)
```

```
[1] 2
```

```
which(x==min(x))
```

```
[1] 1
```

```
x[which(x==min(x))]#Returns the  value
```

```
[1] 2
```

```
which.min(x)#Returns the position,not values
```

```
[1] 1
```

```
x=c(2,3,4,5,11,112,133,33)
max(x)
```

```
[1] 133
```

```r
which(x==max(x)) #Returns the position,not values
```

```
[1] 7
```

```r
x[which(x==max(x))]#Returns the  value
```

```
[1] 133
```

```r
which.max(x)#Returns the position,not values
```

```
[1] 7
```

```r
x=c(8,7,4,5,11,112,133,33)
which(x>2 & x<5)
```

```
[1] 3
```

```r
x[which(x>2 & x<5)]
```

```
[1] 4
```

```r
x=c(8,7,4,5,11,112,133,33)
which(x>7 | x<12)
```

```
[1] 1 2 3 4 5 6 7 8
```

```r
x[which(x>7 | x<12)]
```

```
[1]   8   7   4   5  11 112 133  33
```

```r
#6)"REP" function
x=rep(1:5,times=10)
x=rep(100,times=10)
x=rep(c(3,6),times=4)
x=rep("Kummam",times=5)
x=rep(c("Ramesh","Kummam"),times=3)
```

```r
x=rep(1:4,5:8)
#x=rep(1:4,1:2) #output as invalid argument
x=rep(1:4,c(2,3,5,7))
x=rep(1:4,each=3)
x=rep(1:4,each=2,times=3)

#7)"SEQ" function
x=seq(from=1,to=10,by=3)
#x=seq(from=1,to=10,by=-3) # wrong arguments
x=seq(from=10,to=1,by=-3)
x=seq(from=1,to=10,length=100)
x=seq(from=1,by=2,length=100)
y=seq(from=1,by=3,length=50)
z=c(x,y)

#8)seq_len() & seq_along() functions
x=c(8,7,4,5,11,112,133,33)
length(x)
```

```
[1] 8
```

```r
seq_len(length(x))
```

```
[1] 1 2 3 4 5 6 7 8
```

```r
seq_along(x) #Returns length of the object
```

```
[1] 1 2 3 4 5 6 7 8
```

```r
#9) Dealing with missing values
x=c(11,3,5,7)
x[2]=NA
x[c(2,3)]=NA
is.na(x) #Output is a logical vector
```

```
[1] FALSE  TRUE  TRUE FALSE
```

```
x[!is.na(x)]
```

```
[1] 11  7
```

```
na.omit(x)
```

```
[1] 11  7
attr(,"na.action")
[1] 2 3
attr(,"class")
[1] "omit"
```

```
#Note: Observe the below operations carefully
x=c(1,NA,5,NA)
#x==NA # not followed it so far
#x[x==NA]# not followed it so far


#10) Naming a vector
x=c(1:3)
names(x)=c("a","b","c")
x[c("a","b")]
```

```
a b
1 2
```

```
y=c("Ramesh","Kummam")
names(y)=c("First","Last")
y[c("Last","First")]
```

```
    Last    First
"Kummam" "Ramesh"
```

```
#9) Checking the availability of elements in a vector
a=c(1:10)
b=c(5:15)
1 %in% a
```

```
[1] TRUE
```

```
1 %in% b
```

```
[1] FALSE
```

```
a %in% b
```

```
 [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
b %in% a
```

```
 [1]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE
```

```
is.element(a,b)
```

```
 [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
is.element(b,a)
```

```
 [1]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE
```

```
# print strings
print("string")
```

```
[1] "string"
```

```
# concatenate strings "a" + "b" = "ab"
paste("a", "b", sep = "")
```

```
[1] "ab"
```

```
# paste objects of different length
paste("i", 1:10, sep =".")
```

```
 [1] "i.1"  "i.2"  "i.3"  "i.4"  "i.5"  "i.6"  "i.7"  "i.8"  "i.9"  "i.10"
```

```r
paste(c("i","j", "k"), 1:10, sep =".")
```

```
 [1] "i.1"  "j.2"  "k.3"  "i.4"  "j.5"  "k.6"  "i.7"  "j.8"  "k.9"  "i.10"
```

```r
# paste with collapsing
paste(c("i","j", "k"), 1:3, sep = "", collapse = "")
```

```
[1] "i1j2k3"
```

```r
# paste withour collapsing
paste(c("i","j", "k"), 1:3, sep = "")
```

```
[1] "i1" "j2" "k3"
```

```r
# paste0() shorter version of paste(..., sep ="")
paste0("Hello", "world", ",", "I", "use", "R")
```

```
[1] "Helloworld,IuseR"
```

```r
paste("Hello", "world", ",", "I", "use", "R", sep =" ")
```

```
[1] "Hello world , I use R"
```

```r
# concatenate strings with cat()
cat("Hello", "world", "!")
```

```
Hello world !
```

```r
# it prints withoute "" quotes !!!
cat("Hello", "world", "!", sep = "/")
```

```
Hello/world/!
```

```r
# counting number of characters nchar()
nchar("Hello world")
```

```
[1] 11
```

```r
# load US states names from data frame regarding crime rate
df <- USArrests
head(df)
```

```
           Murder Assault UrbanPop Rape
Alabama      13.2     236       58 21.2
Alaska       10.0     263       48 44.5
Arizona       8.1     294       80 31.0
Arkansas      8.8     190       50 19.5
California    9.0     276       91 40.6
Colorado      7.9     204       78 38.7
```

```r
rownames(df)
```

```
 [1] "Alabama"        "Alaska"         "Arizona"        "Arkansas"
 [5] "California"     "Colorado"       "Connecticut"    "Delaware"
 [9] "Florida"        "Georgia"        "Hawaii"         "Idaho"
[13] "Illinois"       "Indiana"        "Iowa"           "Kansas"
[17] "Kentucky"       "Louisiana"      "Maine"          "Maryland"
[21] "Massachusetts"  "Michigan"       "Minnesota"      "Mississippi"
[25] "Missouri"       "Montana"        "Nebraska"       "Nevada"
[29] "New Hampshire"  "New Jersey"     "New Mexico"     "New York"
[33] "North Carolina" "North Dakota"   "Ohio"           "Oklahoma"
[37] "Oregon"         "Pennsylvania"   "Rhode Island"   "South Carolina"
[41] "South Dakota"   "Tennessee"      "Texas"          "Utah"
[45] "Vermont"        "Virginia"       "Washington"     "West Virginia"
[49] "Wisconsin"      "Wyoming"
```

```r
states <- rownames(df)

# convert all states names to upper case
states_upper <- toupper(states)

# convert all names to lower
states_lower <- tolower(states)

# or select which to apply with function casefol
casefold(x = states, upper = T)
```

```
 [1] "ALABAMA"        "ALASKA"         "ARIZONA"        "ARKANSAS"
```

```
 [5] "CALIFORNIA"     "COLORADO"       "CONNECTICUT"    "DELAWARE"
 [9] "FLORIDA"        "GEORGIA"        "HAWAII"         "IDAHO"
[13] "ILLINOIS"       "INDIANA"        "IOWA"           "KANSAS"
[17] "KENTUCKY"       "LOUISIANA"      "MAINE"          "MARYLAND"
[21] "MASSACHUSETTS"  "MICHIGAN"       "MINNESOTA"      "MISSISSIPPI"
[25] "MISSOURI"       "MONTANA"        "NEBRASKA"       "NEVADA"
[29] "NEW HAMPSHIRE"  "NEW JERSEY"     "NEW MEXICO"     "NEW YORK"
[33] "NORTH CAROLINA" "NORTH DAKOTA"   "OHIO"           "OKLAHOMA"
[37] "OREGON"         "PENNSYLVANIA"   "RHODE ISLAND"   "SOUTH CAROLINA"
[41] "SOUTH DAKOTA"   "TENNESSEE"      "TEXAS"          "UTAH"
[45] "VERMONT"        "VIRGINIA"       "WASHINGTON"     "WEST VIRGINIA"
[49] "WISCONSIN"      "WYOMING"
```

```
casefold(x = states, upper = F)
```

```
 [1] "alabama"        "alaska"         "arizona"        "arkansas"
 [5] "california"     "colorado"       "connecticut"    "delaware"
 [9] "florida"        "georgia"        "hawaii"         "idaho"
[13] "illinois"       "indiana"        "iowa"           "kansas"
[17] "kentucky"       "louisiana"      "maine"          "maryland"
[21] "massachusetts"  "michigan"       "minnesota"      "mississippi"
[25] "missouri"       "montana"        "nebraska"       "nevada"
[29] "new hampshire"  "new jersey"     "new mexico"     "new york"
[33] "north carolina" "north dakota"   "ohio"           "oklahoma"
[37] "oregon"         "pennsylvania"   "rhode island"   "south carolina"
[41] "south dakota"   "tennessee"      "texas"          "utah"
[45] "vermont"        "virginia"       "washington"     "west virginia"
[49] "wisconsin"      "wyoming"
```

```
# character translation
chartr(old = "o", new = "0", x = "Hello World")
```

```
[1] "Hell0 W0rld"
```

```
# sorting strings
sort(states, decreasing = F) #ascending order
```

```
 [1] "Alabama"        "Alaska"         "Arizona"        "Arkansas"
 [5] "California"     "Colorado"       "Connecticut"    "Delaware"
 [9] "Florida"        "Georgia"        "Hawaii"         "Idaho"
```

```
[13] "Illinois"       "Indiana"        "Iowa"           "Kansas"
[17] "Kentucky"       "Louisiana"      "Maine"          "Maryland"
[21] "Massachusetts"  "Michigan"       "Minnesota"      "Mississippi"
[25] "Missouri"       "Montana"        "Nebraska"       "Nevada"
[29] "New Hampshire"  "New Jersey"     "New Mexico"     "New York"
[33] "North Carolina" "North Dakota"   "Ohio"           "Oklahoma"
[37] "Oregon"         "Pennsylvania"   "Rhode Island"   "South Carolina"
[41] "South Dakota"   "Tennessee"      "Texas"          "Utah"
[45] "Vermont"        "Virginia"       "Washington"     "West Virginia"
[49] "Wisconsin"      "Wyoming"
```

```
sort(states, decreasing = T) #descending order
```

```
 [1] "Wyoming"        "Wisconsin"      "West Virginia"  "Washington"
 [5] "Virginia"       "Vermont"        "Utah"           "Texas"
 [9] "Tennessee"      "South Dakota"   "South Carolina" "Rhode Island"
[13] "Pennsylvania"   "Oregon"         "Oklahoma"       "Ohio"
[17] "North Dakota"   "North Carolina" "New York"       "New Mexico"
[21] "New Jersey"     "New Hampshire"  "Nevada"         "Nebraska"
[25] "Montana"        "Missouri"       "Mississippi"    "Minnesota"
[29] "Michigan"       "Massachusetts"  "Maryland"       "Maine"
[33] "Louisiana"      "Kentucky"       "Kansas"         "Iowa"
[37] "Indiana"        "Illinois"       "Idaho"          "Hawaii"
[41] "Georgia"        "Florida"        "Delaware"       "Connecticut"
[45] "Colorado"       "California"     "Arkansas"       "Arizona"
[49] "Alaska"         "Alabama"
```

```
# extracting parts of string
# sub string first 3 letters from state name Alabama
substr(x = "Alabama", start = 1, stop = 3)
```

```
[1] "Ala"
```

```
# String matching - back to toy example
help(regex)

# get all country names
#install.packages("countrycode")
require(countrycode)
```

```
Loading required package: countrycode
```

```
countries <- as.vector(countrycode::codelist$country.name.en)

# countries beginning with letter "A"
grep(pattern = "^A", x = countries)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
```

```
countries[grep(pattern = "^A", x = countries)]
```

```
 [1] "Afghanistan"       "Albania"           "Algeria"
 [4] "American Samoa"    "Andorra"           "Angola"
 [7] "Anguilla"          "Antarctica"        "Antigua & Barbuda"
[10] "Argentina"         "Armenia"           "Aruba"
[13] "Australia"         "Austria"           "Austria-Hungary"
[16] "Azerbaijan"
```

```
countries[grepl(pattern = "^A", x = countries)]
```

```
 [1] "Afghanistan"       "Albania"           "Algeria"
 [4] "American Samoa"    "Andorra"           "Angola"
 [7] "Anguilla"          "Antarctica"        "Antigua & Barbuda"
[10] "Argentina"         "Armenia"           "Aruba"
[13] "Australia"         "Austria"           "Austria-Hungary"
[16] "Azerbaijan"
```

```
# all country names that end with letter "y"
rez <- grep(pattern = "*y$", x = countries)
countries[rez]
```

```
 [1] "Austria-Hungary"                "British Indian Ocean Territory"
 [3] "Germany"                        "Guernsey"
 [5] "Hungary"                        "Italy"
 [7] "Jersey"                         "Norway"
 [9] "Paraguay"                       "Saxony"
[11] "St. Barthélemy"                 "Turkey"
[13] "Tuscany"                        "Uruguay"
[15] "Vatican City"
```

```r
# all country with 2 words or more for a country name
rez <- grep(pattern = "\\w\\s\\w", x = countries)
countries[rez]
```

```
 [1] "American Samoa"
 [2] "Bouvet Island"
 [3] "British Indian Ocean Territory"
 [4] "British Virgin Islands"
 [5] "Burkina Faso"
 [6] "Cape Verde"
 [7] "Caribbean Netherlands"
 [8] "Cayman Islands"
 [9] "Central African Republic"
[10] "Channel Islands"
[11] "Christmas Island"
[12] "Cook Islands"
[13] "Costa Rica"
[14] "Côte d'Ivoire"
[15] "Dominican Republic"
[16] "El Salvador"
[17] "Equatorial Guinea"
[18] "Falkland Islands"
[19] "Faroe Islands"
[20] "French Guiana"
[21] "French Polynesia"
[22] "French Southern Territories"
[23] "German Democratic Republic"
[24] "Heard & McDonald Islands"
[25] "Hesse Electoral"
[26] "Hesse Grand Ducal"
[27] "Hong Kong SAR China"
[28] "Isle of Man"
[29] "Macao SAR China"
[30] "Marshall Islands"
[31] "Mecklenburg Schwerin"
[32] "Micronesia (Federated States of)"
[33] "Netherlands Antilles"
[34] "New Caledonia"
[35] "New Zealand"
[36] "Norfolk Island"
[37] "North Korea"
[38] "North Macedonia"
```

```
[39] "Northern Mariana Islands"
[40] "Orange Free State"
[41] "Palestinian Territories"
[42] "Papua New Guinea"
[43] "Pitcairn Islands"
[44] "Puerto Rico"
[45] "Republic of Vietnam"
[46] "Saint Martin (French part)"
[47] "San Marino"
[48] "Saudi Arabia"
[49] "Serbia and Montenegro"
[50] "Sierra Leone"
[51] "Sint Maarten"
[52] "Solomon Islands"
[53] "South Africa"
[54] "South Georgia & South Sandwich Islands"
[55] "South Korea"
[56] "South Sudan"
[57] "Sri Lanka"
[58] "Svalbard & Jan Mayen"
[59] "São Tomé & Príncipe"
[60] "Turks & Caicos Islands"
[61] "Two Sicilies"
[62] "U.S. Virgin Islands"
[63] "United Arab Emirates"
[64] "United Arab Republic"
[65] "United Kingdom"
[66] "United Province CA"
[67] "United States"
[68] "United States Minor Outlying Islands (the)"
[69] "Vatican City"
[70] "Western Sahara"
[71] "Yemen Arab Republic"
[72] "Yemen People's Republic"
[73] "Åland Islands"
```

```
# all country names that end with letter "e" or "i"
rez <- grep(pattern = "*e$|*i$", x = countries)
countries[rez]
```

```
 [1] "Belize"              "Brunei"              "Burundi"
 [4] "Cape Verde"          "Chile"               "Congo - Brazzaville"
```

```
 [7] "Côte d'Ivoire"       "Djibouti"            "Eswatini"
[10] "Fiji"                "France"              "Greece"
[13] "Guadeloupe"          "Haiti"               "Kiribati"
[16] "Malawi"              "Mali"                "Martinique"
[19] "Mayotte"             "Mozambique"          "Niue"
[22] "Orange Free State"   "Sierra Leone"        "Singapore"
[25] "Suriname"            "São Tomé & Príncipe" "Timor-Leste"
[28] "Ukraine"             "Zimbabwe"
```

```r
# all country names which contain combination of letters "gin"
rez <- grep(pattern = "*(gin)", x = countries)
countries[rez]
```

```
[1] "British Virgin Islands" "U.S. Virgin Islands"
```

```r
# metacharacters and double backslash sign
strings <- c("dollar $", "dollar", "US dollar")

# how to escape metacharacters in R
# we would like to match a word with "$" dollar sign
strings
```

```
[1] "dollar $"  "dollar"    "US dollar"
```

```r
rez1 <- grep(pattern = "$", x = strings) # wrong way all are find $ indicating end of string
strings[rez1]
```

```
[1] "dollar $"  "dollar"    "US dollar"
```

```r
# we need to escape $ with \ backslash
#rez2 <- grep(pattern = "\$", x = strings) # wrong way error
#strings[rez2]

# right way $ escape with \\ double backslash
#rez3 <- grep(pattern = "\\$", x = strings) # wrong way error
#strings[rez3]

# Escape dot .
strings <-c("word.word", "word word")
rez <- grep(pattern = "\\.", x = strings)
strings[rez]
```

```
[1] "word.word"
```

```
# the hat sign beginning of the string and dollar sign end of string
strings <- c("Word", "worD")
strings[grep("^W",strings)]
```

```
[1] "Word"
```

```
strings[grep("D$",strings)]
```

```
[1] "worD"
```

```
# example of an anchor sequences in R
strings <- c("123", "onetwothree", "1twothree")
strings[grep("\\d", strings)] # digit character
```

```
[1] "123"       "1twothree"
```

```
strings[grep("\\D", strings)] # non-digit character
```

```
[1] "onetwothree" "1twothree"
```

```
# example of a character classes
strings <- c("123", "dollar", "shkjl")
strings[grep("[aeiou]", strings)] # match word with any vovel
```

```
[1] "dollar"
```

```
strings[grep("[0-9]", strings)] # match word with digits
```

```
[1] "123"
```

```
# find and replace sub() & gsub()
string <- "I have started to learn R, and I already love R."

# replace "R" with "X"
sub(pattern = "R", replacement = "X", x = string) #only first occurence replaced
```

```
[1] "I have started to learn X, and I already love R."
```

```r
gsub(pattern = "R", replacement = "X", x = string) #all occurences replaced
```

```
[1] "I have started to learn X, and I already love X."
```

```r
#replace white space with blank space
gsub(pattern = "\\s", replacement = "", x = string)
```

```
[1] "IhavestartedtolearnR,andIalreadyloveR."
```

```r
# string split, split given sentence by comma ","
strsplit(x = string, split = ",")
```

```
[[1]]
[1] "I have started to learn R" " and I already love R."
```

```r
# split phone numbers by digits
numbers <- c("310-555-123", "311-444-456")
strsplit(x = numbers, split = "-")
```

```
[[1]]
[1] "310" "555" "123"

[[2]]
[1] "311" "444" "456"
```

24 Data Frames with dplyr

```
# create data frame
df1 <- data.frame(col1 = 1:3,
                  col2 = c("a", "b", "c"),
                  col3 = c(T, F, T),
                  col4 = c(as.Date("2020-01-01"), as.Date("2020-01-03"), as.Date("2020-01-03"


# create data frame - vectors
col1 <- seq(10,100,10)
col2 <- seq(as.Date("2020-01-01"), length = 10, by = "weeks")
col3 <- rep("word", 10)

df2 <- data.frame(num = col1,
                  date = col2,
                  string = col3)

# check DF structure
str(df2)
```

```
'data.frame':   10 obs. of  3 variables:
 $ num   : num  10 20 30 40 50 60 70 80 90 100
 $ date  : Date, format: "2020-01-01" "2020-01-08" ...
 $ string: chr  "word" "word" "word" "word" ...
```

```
# create data frame - matrix
M <- matrix(data = 1:100, nrow = 10, ncol = 10, byrow = T)
rownames(M) <- paste("row", 1:10, sep = "")
colnames(M) <- paste("col", 1:10, sep = "")
M
```

```
      col1 col2 col3 col4 col5 col6 col7 col8 col9 col10
row1     1    2    3    4    5    6    7    8    9    10
row2    11   12   13   14   15   16   17   18   19    20
```

```
row3    21   22   23   24   25   26   27   28   29    30
row4    31   32   33   34   35   36   37   38   39    40
row5    41   42   43   44   45   46   47   48   49    50
row6    51   52   53   54   55   56   57   58   59    60
row7    61   62   63   64   65   66   67   68   69    70
row8    71   72   73   74   75   76   77   78   79    80
row9    81   82   83   84   85   86   87   88   89    90
row10   91   92   93   94   95   96   97   98   99   100
```

```
df3 <- as.data.frame(M)
df3
```

```
      col1 col2 col3 col4 col5 col6 col7 col8 col9 col10
row1     1    2    3    4    5    6    7    8    9    10
row2    11   12   13   14   15   16   17   18   19    20
row3    21   22   23   24   25   26   27   28   29    30
row4    31   32   33   34   35   36   37   38   39    40
row5    41   42   43   44   45   46   47   48   49    50
row6    51   52   53   54   55   56   57   58   59    60
row7    61   62   63   64   65   66   67   68   69    70
row8    71   72   73   74   75   76   77   78   79    80
row9    81   82   83   84   85   86   87   88   89    90
row10   91   92   93   94   95   96   97   98   99   100
```

```
# check DF dimensions
dim(df3)
```

```
[1] 10 10
```

```
nrow(df3)
```

```
[1] 10
```

```
ncol(df3)
```

```
[1] 10
```

```
# check DF type / class
class(df3)
```

```
[1] "data.frame"
```

```
typeof(df3)
```

```
[1] "list"
```

```
# 4.3.2 Accessing DF

# let's create DF - employees
df_emp <- data.frame(id = 1:6,
                     name = c("Max", "Jane", "John", "Tony", "Janis", "Helen"),
                     surname = c("Gordon", "Smith", "Don", "Price", "Jett", "Dust"),
                     age = c(55, 35, 46, 22, 60, 27),
                     date_start_work = c(as.Date("1985-09-01"), as.Date("2010-10-01"), as.Da
                     gender = c("M", "F", "M", "M", "F", "M"),
                     manager_position = c(T, F, F, F, T, F)
                     )

# extract data as data frame (one column) - []
df_extr <- df_emp["name"]
df_extr
```

```
   name
1   Max
2  Jane
3  John
4  Tony
5 Janis
6 Helen
```

```
class(df_extr)
```

```
[1] "data.frame"
```

```
# extract data as vector (one column) [[]] $
df_extr <- df_emp[["age"]]
df_extr
```

```
[1] 55 35 46 22 60 27
```

```r
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```r
df_extr <- df_emp$age
df_extr
```

```
[1] 55 35 46 22 60 27
```

```r
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```r
# extract multiple columns
df_extr <- df_emp[c("name", "age")]
df_extr
```

```
   name age
1   Max  55
2  Jane  35
3  John  46
4  Tony  22
5 Janis  60
6 Helen  27
```

```r
# data frame slicing
df_emp
```

```
  id  name surname age date_start_work gender manager_position
1  1   Max  Gordon  55      1985-09-01      M             TRUE
2  2  Jane   Smith  35      2010-10-01      F            FALSE
3  3  John     Don  46      1999-06-01      M            FALSE
4  4  Tony   Price  22      2019-03-01      M            FALSE
5  5 Janis    Jett  60      1980-04-15      F             TRUE
6  6 Helen    Dust  27      2015-02-20      M            FALSE
```

```r
#extract second row in name column (1 cell)
df_emp[2,2]
```

```
[1] "Jane"
```

```r
df_emp[2,"name"]
```

```
[1] "Jane"
```

```r
# extract first 4 rows of last 2 columns
df_emp[1:4, 6:7]
```

```
  gender manager_position
1      M             TRUE
2      F            FALSE
3      M            FALSE
4      M            FALSE
```

```r
df_emp[1:4, c("gender", "manager_position")]
```

```
  gender manager_position
1      M             TRUE
2      F            FALSE
3      M            FALSE
4      M            FALSE
```

```r
# extract first column (all rows)
df_emp[,1]
```

```
[1] 1 2 3 4 5 6
```

```r
df_emp[,"id"]
```

```
[1] 1 2 3 4 5 6
```

```r
df_emp$id
```

```
[1] 1 2 3 4 5 6
```

```r
# extract last 2 rows (all columns)
df_emp[5:6,]
```

```
  id  name surname age date_start_work gender manager_position
5  5 Janis    Jett  60      1980-04-15      F             TRUE
6  6 Helen    Dust  27      2015-02-20      M            FALSE
```

```r
cols <- colnames(df_emp)
df_emp[5:6, cols]
```

```
  id  name surname age date_start_work gender manager_position
5  5 Janis    Jett  60      1980-04-15      F             TRUE
6  6 Helen    Dust  27      2015-02-20      M            FALSE
```

```r
# 4.3.3 Modifying data frame

# append column
df_emp <- cbind(df_emp, role = c("director", "secretary", "analyst", "researcher", "CEO", "a
df_emp$new_col <- 1

# append rows
df_emp <- rbind(df_emp, list(7, "Mark", "Jax", 32, as.Date("2020-01-01"), "M", F, "researche

# problem with factor variables (new values not in factor levels)
# easy solution - append new row as data frame (rbind 2 data frames)!!!
# will show few rows later

# remove column
df_emp$new_col <- NULL

# remove row
df_emp <- df_emp[-7,]

# merge two data frames (row wise)
df_new_emp <- data.frame(id = 7,
```

```
                          name = "Mark",
                          surname = "Jax",
                          age = 32,
                          date_start_work = as.Date("2020-01-01"),
                          gender = "M",
                          manager_position = F,
                          role = "researcher")

df_emp <- rbind(df_emp, df_new_emp)

# merge two data frames (column wise)
df_attr <- data.frame(eye_color = c("blue", "green", "brown", "hazel", "blue", "brown", "brow
                      hair_color = c("blonde", "light brown", "black", "brown", "blonde", "da
df_emp <- cbind(df_emp, df_attr)


# 4.3.4 Tips

# Df summary
summary(df_emp)
```

```
       id          name              surname               age
 Min.   :1.0   Length:7           Length:7           Min.   :22.00
 1st Qu.:2.5   Class :character   Class :character   1st Qu.:29.50
 Median :4.0   Mode  :character   Mode  :character   Median :35.00
 Mean   :4.0                                         Mean   :39.57
 3rd Qu.:5.5                                         3rd Qu.:50.50
 Max.   :7.0                                         Max.   :60.00
 date_start_work         gender          manager_position     role
 Min.   :1980-04-15   Length:7           Mode :logical    Length:7
 1st Qu.:1992-07-16   Class :character   FALSE:5          Class :character
 Median :2010-10-01   Mode  :character   TRUE :2          Mode  :character
 Mean   :2004-05-06
 3rd Qu.:2017-02-24
 Max.   :2020-01-01
  eye_color          hair_color
 Length:7           Length:7
 Class :character   Class :character
 Mode  :character   Mode  :character
```

```r
# rows subsetting
subset(x = df_emp, gender == "M")
```

```
  id  name surname age date_start_work gender manager_position       role
1  1   Max  Gordon  55      1985-09-01      M             TRUE   director
3  3  John     Don  46      1999-06-01      M            FALSE    analyst
4  4  Tony   Price  22      2019-03-01      M            FALSE researcher
6  6 Helen    Dust  27      2015-02-20      M            FALSE    analyst
7  7  Mark     Jax  32      2020-01-01      M            FALSE researcher
  eye_color hair_color
1      blue     blonde
3     brown      black
4     hazel      brown
6     brown dark brown
7     brown      brown
```

```r
subset(x = df_emp, gender == "F" & manager_position == T)
```

```
  id  name surname age date_start_work gender manager_position role eye_color
5  5 Janis    Jett  60      1980-04-15      F             TRUE  CEO      blue
  hair_color
5     blonde
```

```r
rows <- which(df_emp[,"gender"] == "M")
df_emp[rows,]
```

```
  id  name surname age date_start_work gender manager_position       role
1  1   Max  Gordon  55      1985-09-01      M             TRUE   director
3  3  John     Don  46      1999-06-01      M            FALSE    analyst
4  4  Tony   Price  22      2019-03-01      M            FALSE researcher
6  6 Helen    Dust  27      2015-02-20      M            FALSE    analyst
7  7  Mark     Jax  32      2020-01-01      M            FALSE researcher
  eye_color hair_color
1      blue     blonde
3     brown      black
4     hazel      brown
6     brown dark brown
7     brown      brown
```

```
rows <- which(df_emp[,"gender"] == "F" & df_emp[,"manager_position"] == T)
df_emp[rows,]
```

```
  id  name surname age date_start_work gender manager_position role eye_color
5  5 Janis    Jett  60      1980-04-15      F             TRUE  CEO      blue
  hair_color
5     blonde
```

```
# some calculations regarding data frames
nr_managers <- sum(df_emp$manager_position)
mean_age <- mean(df_emp$age)
df_emp$name_surname <- paste(df_emp$name, df_emp$surname, sep = " ") # merge name and surname

# use apply to sum over columns (age, manager_position)
apply(df_emp[,c("age", "manager_position")], 2, sum)
```

```
             age manager_position
             277                2
```

```
# if statement

x <- 3
y <- 14

if(x < y){
  print("x is less than y")
}
```

```
[1] "x is less than y"
```

```
# if-else statement

x <- 3
y <- 14

if(x > y){
  print("x is greater than y")
} else{
  print("x is less or equal to y")
}
```

```
[1] "x is less or equal to y"
```

```
# if-else if-else statement

x <- 14
y <- 14

if(x > y){
  print("x is greater than y")
} else if (x < y){
  print("x is less than y")
} else{
  print("x is equal to y")
}
```

```
[1] "x is equal to y"
```

```
# 5.1.2 relational operators

# scalar
x <- 3

x == 4
```

```
[1] FALSE
```

```
x > 0
```

```
[1] TRUE
```

```
x < 5
```

```
[1] TRUE
```

```
x <= 3
```

```
[1] TRUE
```

```r
x >= 10
```

```
[1] FALSE
```

```r
x %in% c(1,2,3,4,5)
```

```
[1] TRUE
```

```r
# vectors
X <- c(F,T,0,10)
Y <- c(T,F,F,T)

X == Y # element-wise comparison
```

```
[1] FALSE FALSE  TRUE FALSE
```

```r
X > Y
```

```
[1] FALSE  TRUE FALSE  TRUE
```

```r
X < Y
```

```
[1]  TRUE FALSE FALSE FALSE
```

```r
X >= Y
```

```
[1] FALSE  TRUE  TRUE  TRUE
```

```r
X <= Y
```

```
[1]  TRUE FALSE  TRUE FALSE
```

```r
X != Y
```

```
[1]  TRUE  TRUE FALSE  TRUE
```

```
X %in% Y
```

```
[1]  TRUE  TRUE  TRUE FALSE
```

```
# 5.1.3 Logical operators

X | Y
```

```
[1]  TRUE  TRUE FALSE  TRUE
```

```
#X || Y
X & Y
```

```
[1] FALSE FALSE FALSE  TRUE
```

```
#X && Y
!X
```

```
[1]  TRUE FALSE  TRUE FALSE
```

```
# logical operators in if statement (flip coin twice)
c1 <- "H"
c2 <- "H"

if(c1 == "H" & c2 == "H"){
  paste("You win 10$")
} else if((c1 == "H" & c2 == "T") | (c1 == "T" & c2 == "H")){
  paste("You win 2$")
} else{
  paste("You lose all the money")
}
```

```
[1] "You win 10$"
```

```
# loop over iterations and print number of iteration
for(it in 1:10){
  print(paste("iteration ", it, sep = ""))
}
```

```
[1] "iteration 1"
[1] "iteration 2"
[1] "iteration 3"
[1] "iteration 4"
[1] "iteration 5"
[1] "iteration 6"
[1] "iteration 7"
[1] "iteration 8"
[1] "iteration 9"
[1] "iteration 10"
```

```r
# sum of numbers in a sequence
sequence <- c(1,2,3,4,5)
s <- 0 # initial sum

for(val in sequence){
  s <- s + val
  print(paste("value = ", val, sep = ""))
  print(paste("s = ", s, sep = ""))
  print("--------------------")
}
```

```
[1] "value = 1"
[1] "s = 1"
[1] "--------------------"
[1] "value = 2"
[1] "s = 3"
[1] "--------------------"
[1] "value = 3"
[1] "s = 6"
[1] "--------------------"
[1] "value = 4"
[1] "s = 10"
[1] "--------------------"
[1] "value = 5"
[1] "s = 15"
[1] "--------------------"
```

```r
# for loop with conditional statement
# (count number of even numbers in a vector)
x <- c(1,3,2,4,5,10,22,21,100) # given numbers
count <- 0 # counter for even numbers
```

```r
for(val in x){
  if(val %% 2 == 0){ # if number is divisible with 2
    count <- count + 1
  }
}

print(count)
```

```
[1] 5
```

25 Read sas dataset

```
dm <- haven::read_sas("data/sdtm/dm.sas7bdat")
ae <- haven::read_sas("data/sdtm/ae.sas7bdat")
```

26 keeping selected columns

```
dm_1 <- dm |>
       dplyr::select(USUBJID,ARM) |>
       dplyr::filter(ARM =="Placebo")

dm_2 <- dm |>
  dplyr::select(USUBJID:ARM)

dm_2 <- dm |>
  dplyr::select(1,2,3)

dm_2 <- dm |>
  dplyr::select(1:3)

dm_2 <- dm |>
  dplyr::select(starts_with("AR"))

dm_2 <- dm |>
  dplyr::select(contains("DT"))


dm2 <- dplyr::select(dm,USUBJID,ARM)
```

27 dropping columns

To select all columns except certain ones, put a "-" in front of the variable to exclude it. For multiple variables, you can use the function c() to combine values into a vector or list. This will select all the variables in surveys except ARM, SITEID

```
adsl <- haven::read_sas("data/adam/adsl.sas7bdat")
adsl2 <- dplyr::select(adsl, -c(AGE, SEX, TRT01A))
adsl3 <- dplyr::select(adsl, -c(STUDYID:ARM))
adsl4 <- dplyr::select(adsl, -c(1:6))
adsl4 <- dplyr::select(adsl, -c(1:6, 43))
adsl5 <- dplyr::select(adsl, -starts_with("A"))
adsl6 <- dplyr::select(adsl, -ends_with("DTC"))
adsl7 <- dplyr::select(adsl, -contains("TRT"))
```

There are many helper functions available with select() like: starts_with(), ends_with(), contains() among others. These were imported from the tidyselect package. You can put a "-" in front of the helper function to negate it. Here is an example using contains(): # Filtering rows To choose rows based on a specific criteria, use filter(): To select rows where the planned treatment code equals "Placebo":

```
adsl1 <- adsl |>
     dplyr::select(USUBJID,TRT01A) |>
     dplyr::filter(TRT01A=="Placebo" )

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & SEX == "M" & AGE == 70)

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & (SEX == "M" | AGE == 70))

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & SEX == "M" & AGE %in% c(70,80))
```

```r
adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & !SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A %in% c("Placebo","Xanomeline High Dose") & !SEX == "M" & !AGE %in% c(

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A %in% c("Placebo","Xanomeline High Dose") & !SEX == "M" & AGE >= 70)
```

28 Rename

```
adsl1 <- adsl |>
    dplyr::rename(usubjid=USUBJID)

adsl1 <- adsl |>
  dplyr::rename(usubjid=USUBJID,
                trt=TRT01P)
```

29 Arrange (sorting)

```
adsl1 <- adsl |>
      dplyr::select(USUBJID,SEX,AGE) |>
      dplyr::arrange(AGE,SEX)

adsl1 <- adsl |>
  dplyr::select(USUBJID,SEX,AGE) |>
  dplyr::arrange(AGE,desc(SEX))
```

30 convert var names to lower case

```
tolower(names(adsl))
```

```
 [1] "studyid"  "usubjid"  "subjid"   "siteid"   "sitegr1"  "arm"
 [7] "trt01p"   "trt01pn"  "trt01a"   "trt01an"  "trtsdt"   "trtedt"
[13] "trtdur"   "avgdd"    "cumdose"  "age"      "agegr1"   "agegr1n"
[19] "ageu"     "race"     "racen"    "sex"      "ethnic"   "saffl"
[25] "ittfl"    "efffl"    "comp8fl"  "comp16fl" "comp24fl" "disconfl"
[31] "dsraefl"  "dthfl"    "bmibl"    "bmiblgr1" "heightbl" "weightbl"
[37] "educlvl"  "disonsdt" "durdis"   "durdsgr1" "visit1dt" "rfstdtc"
[43] "rfendtc"  "visnumen" "rfendt"   "dcdecod"  "dcreascd" "mmsetot"
```

```
adsl1 <- adsl

names(adsl1) <- tolower(names(adsl1))
```

31 Creating new variables

```
adsl1 <- adsl |>
  dplyr::mutate(name="Sri Ram")

adsl_test <- adsl |>
  dplyr::mutate(
    AGEGR1_t = dplyr::if_else(AGE < 65, "<65", ">=65"),                    # character
    SAFFL_t  = dplyr::if_else(!is.na(TRTSDT), "Y", "N")                   # character flag
  ) |>
  dplyr::select(USUBJID,AGE,AGEGR1_t,SAFFL_t,TRTSDT)
```

32 bind rows (set operator in SAS)

```r
# bind rows
df_a <- data.frame(
  id = 1:3,
  value = c("A", "B", "C"),
  score = c(10, 20, 30)
)
df_b <- data.frame(
  id = 4:6,
  value = c("D", "E", "F"),
  score = c(40, 50, 60)
)

com <- dplyr::bind_rows(df_a,
                        df_b)

df_combined <- dplyr::bind_rows(df_a, df_b,df_a,.id = "source")

df_combined <- dplyr::bind_rows(df_a, df_b,df_a,.id = "source")

df_combined <- dplyr::bind_rows("dset1"=df_a,
                                "dset2"=df_b,
                                "test"=df_a,.id = "Source_var_name")

df_combined <- dplyr::bind_rows(select(df_a,id, value),
                                df_b)

adsl_tot <- dplyr::bind_rows(adsl |>
                               dplyr::select(USUBJID,TRT01P,TRT01PN),

                             adsl |>
                               dplyr::mutate(TRT01P="Total", TRT01PN=99) |>
                               dplyr::select(USUBJID,TRT01P,TRT01PN)
)
```

33 recode (similar to PROC FORMAT in SAS)

```
sex_fm <- c("M"="Male",
            "F"="Female"
)
adsl_test1 <- adsl |>
      dplyr::mutate(gender=dplyr::recode(SEX,!!!sex_fm))

adsl_test2 <- adsl |>
        dplyr::mutate(SEX=dplyr::recode(SEX,"M"="Male",
                                            "F"="Female"))

adsl_test <- adsl |>
  dplyr::mutate(
    gn = dplyr::if_else(SEX=="M","Male","Female")                    # character
                ) |>
  dplyr::select(USUBJID,SEX,gn) |>
  head(5)
```

34 joins

The dplyr package provides functions to join tables based on shared keys, such as patient IDs. The main types of joins include:

- inner_join()
- left_join()
- right_join()
- full_join()
- semi_join()
- anti_join()

```
# Patient demographics data
patients <- data.frame(
  patient_id = c(101, 102, 103, 104),
  age = c(34, 45, 29, 56),
  gender = c("F", "M", "M", "F")
)

# Patient lab results data
lab_results <- data.frame(
  patient_id = c(103, 104, 105, 106),
  test = c("CBC", "Lipid Panel", "Blood Glucose", "CBC"),
  result = c("Normal", "High Cholesterol", "Elevated", "Normal")
)

# Display the data frames
patients
```

```
  patient_id age gender
1        101  34      F
2        102  45      M
3        103  29      M
4        104  56      F
```

```
lab_results
```

```
  patient_id          test           result
1        103           CBC           Normal
2        104   Lipid Panel High Cholesterol
3        105 Blood Glucose         Elevated
4        106           CBC           Normal
```

34.1 innter join

`inner_join()`: returns only rows with matching patient IDs in both data frames.

```
# Inner join on patient_id
inner_join(patients, lab_results, by = "patient_id")
```

```
  patient_id age gender        test           result
1        103  29      M         CBC           Normal
2        104  56      F Lipid Panel High Cholesterol
```

In this example, only patients with patient_ids 103 and 104 are in both tables, so only their information appears in the result.

34.2 left join

`left_join()` returns all rows from the patients data frame and matches rows from lab_results where possible. Unmatched rows in lab_results are filled with `NA`.

```
# Left join on patient_id
left_join(patients, lab_results, by = "patient_id")
```

```
  patient_id age gender        test           result
1        101  34      F        <NA>             <NA>
2        102  45      M        <NA>             <NA>
3        103  29      M         CBC           Normal
4        104  56      F Lipid Panel High Cholesterol
```

Here, all patients from patients are included, with `NA` for lab results where no match is found.

34.3 right join

`right_join()` returns all rows from lab_results, matching rows from patients where possible. Unmatched rows in patients are filled with`NA`.

```
# Right join on patient_id
right_join(patients, lab_results, by = "patient_id")
```

```
  patient_id age gender          test           result
1        103  29      M           CBC           Normal
2        104  56      F   Lipid Panel High Cholesterol
3        105  NA   <NA> Blood Glucose         Elevated
4        106  NA   <NA>           CBC           Normal
```

In this example, all patients with lab results are included, with demographic data from patients where available.

34.4 Full Join

`full_join()` returns all rows from both tables, with `NA` for missing matches in either table.

```
# Full join on patient_id
full_join(patients, lab_results, by = "patient_id")
```

```
  patient_id age gender          test           result
1        101  34      F          <NA>             <NA>
2        102  45      M          <NA>             <NA>
3        103  29      M           CBC           Normal
4        104  56      F   Lipid Panel High Cholesterol
5        105  NA   <NA> Blood Glucose         Elevated
6        106  NA   <NA>           CBC           Normal
```

34.5 Semi Join

`semi_join()` returns only the rows in patients that have a match in lab_results, without bringing in columns from lab_results.

```r
# Semi join on patient_id
semi_join(patients, lab_results, by = "patient_id")
```

```
  patient_id age gender
1        103  29      M
2        104  56      F
```

Only patients with lab results are included here (patients 103 and 104).

34.6 Anti join

`anti_join()` returns only the rows in patients that do not have a match in lab_results.

```r
# Anti join on patient_id
anti_join(patients, lab_results, by = "patient_id")
```

```
  patient_id age gender
1        101  34      F
2        102  45      M
```

```r
#example
tab_a <- data.frame(
  id=c(001,002,003,004),
  age=c(25,50,30,40)
)

tab_b <- data.frame(
  id=c(001,002,004,005),
  sex=c("M","F","M","F")
)

#inner join:
injointab <- dplyr::inner_join(tab_a,tab_b,by="id")
#full join:
fulljointab <- full_join(tab_a,tab_b,by="id")
#left join
leftjointab <- left_join(x=tab_a,y=tab_b,by="id")
#right join
righjointab <- right_join(x=tab_a,y=tab_b,by="id")
```

```
#anti_join
antijointab <- anti_join(x=tab_a,y=tab_b,by="id")
antijointab1 <- anti_join(x=tab_b,y=tab_a,by="id")
```

35 Reshaping data with tidyr

```
test <- data.frame(
  visit=c("wk0","wk2","wk4","wk6","wk12"),
  drug_a=c(10,20,30,40,50),
  drug_b=c(50,40,30,20,10),
  drug_c=c(15,25,35,45,55))
#tidyr::pivot_longer
library(tidyr)
test1 <- pivot_longer(data=test,
                      cols=c(drug_a,drug_b,drug_c),
                      names_to="drug",
                      values_to = "dose")


test2 <- pivot_wider(data=test1,
                     id_cols =visit,
                     names_prefix = "d_", # it is optional
                     values_from = dose,
                     names_from = drug)
```

36 counting with `n()`

When working with data, we often want to know the number of observations found for each factor or combination of factors. For this task, dplyr provides count(). For example, if we wanted to count the number of rows of data for each sex, we would do:

```
cnt <- adsl |>
  dplyr::count(SEX)


# race count by treatment
cnt2 <- adsl |>
  dplyr::count(TRT01P, RACE)


#treatment count
cnt3 <- adsl |>
  dplyr::filter(!is.na(TRT01P)) |>
  dplyr::count(TRT01P)

adae <- haven::read_sas("data/adam/adae.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

adsl <- haven::read_sas("data/adam/adsl.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

bign <- adsl |>
  dplyr::filter(SAFFL == "Y") |>
  dplyr::count(TRT01AN, TRT01A) |>
  dplyr::rename(TRTA = TRT01A, TRTAN = TRT01AN, bign = n)

bign1 <- dplyr::count(adsl, TRT01AN, TRT01A) |>
  dplyr::rename(TRTA = TRT01A, TRTAN = TRT01AN, bign = n)

adae1 <- adae |>
   dplyr::distinct(USUBJID,AEBODSYS,AEDECOD,TRTA,TRTAN,SAFFL)
```

```
ae_t <- adae1 |>
  dplyr::filter(SAFFL == "Y") |>
  dplyr::group_by(TRTA) |>
  dplyr::count(AEBODSYS, AEDECOD)

ae_t1 <- ae_t |>
  dplyr::left_join(bign, by = "TRTA") |>
  dplyr::mutate(
    pct = paste0("(", round(n / bign * 100, 2), ")"),
    val = paste0(n," ",pct)
  )

ae2 <- pivot_wider(
  data = ae_t1,
  id_cols = c(AEBODSYS, AEDECOD),
  names_from = TRTA,
  values_from = val,
  values_fill = "0 (0.0)"
)
```

37 dplyr distinct()

The distinct() function in dplyr is used to return unique/distinct rows from a data frame or tibble. It removes duplicate rows based on the values in specified columns or all columns if none are specified.

```
# distinct
distinct_trt <- adsl |>
  dplyr::distinct(TRT01P,TRT01PN)
print(distinct_trt)
```

```
# A tibble: 3 x 2
  TRT01P               TRT01PN
  <chr>                  <dbl>
1 Placebo                    0
2 Xanomeline High Dose      81
3 Xanomeline Low Dose       54
```

38 case_when()

The case_when() function in dplyr is a powerful tool for creating new variables based on multiple conditions. It allows you to specify a series of conditions and corresponding values to assign when those conditions are met.

```
adsl1 <- adsl |>
  dplyr::mutate(
    agegrp1 = dplyr::case_when(
      AGE < 18 ~ "Child",
      AGE >= 18 & AGE < 65 ~ "Adult",
      AGE >= 65 ~ "Senior",
      TRUE ~ "Unknown"  # Default case
    )
  ) |>
  dplyr::select(USUBJID, AGE, agegrp1) |>
  head(10)
```

39 create seq no.

```
class <- haven::read_sas("data/class.sas7bdat")


t <- dplyr::arrange(class, Age)
# Create t1 data set with first and last records for each unique value of 'age'

t1 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
     dplyr::mutate(seq = dplyr::row_number())

t2 <- class |>
  dplyr::mutate(seq = dplyr::row_number(), .by = Age) |>
  dplyr::arrange(Age)

t3 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::filter(dplyr::row_number() == 1 )

t4 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::filter(dplyr::row_number() == n())

t5 <- class |>
  dplyr::group_by(Age) |>
  dplyr::filter(row_number() == 1 & row_number() == n())

t6 <- class |>
  dplyr::group_by(Age) |>
  dplyr::filter(row_number() == 1|row_number() == n())
```

40 Lists: `lapply`, purrr

```r
lst <- list(a=1:3, b=10:12)
lapply(lst, mean)
```

```
$a
[1] 2

$b
[1] 11
```

41 Exercises

1. Using `across()`, standardize (`(x-mean)/sd`) numeric columns.
2. Create a row-wise mean of `age` and `wt`.
3. Split `df` by `grp` and compute group means with `lapply` or `purrr::map`.

42 String Functions

43 String Functions in Base R

Strings (or character vectors) are a fundamental data type in R, and base R provides a variety of functions to manipulate and analyze strings. Here are some commonly used string functions in base R:

```
# Create a character vector
text <- c("Hello, World!", "R is great", "Data Science")
```

43.1 Basic String Functions

```
# nchar(): Returns the number of characters in a string
nchar(text)
```

```
[1] 13 10 12
```

```
# tolower(): Converts a string to lowercase
tolower(text)
```

```
[1] "hello, world!" "r is great"    "data science"
```

```
# toupper(): Converts a string to uppercase
toupper(text)
```

```
[1] "HELLO, WORLD!" "R IS GREAT"    "DATA SCIENCE"
```

```
# substr(): Extracts a substring from a string
substr(text, start = 1, stop = 5)
```

```
[1] "Hello" "R is " "Data "
```

```r
# paste(): Concatenates strings together
paste("Hello", "R", sep = " ")
```

```
[1] "Hello R"
```

```r
# paste0(): Concatenates strings without any separator
paste0("Hello", "R")
```

```
[1] "HelloR"
```

```r
# trimws(): Trims leading and trailing whitespace from a string
trimws("   Hello, World!   ")
```

```
[1] "Hello, World!"
```

43.2 Pattern Matching and Replacement

```r
# grep(): Searches for patterns in a character vector and returns the indices of matches
grep("R", text)
```

```
[1] 2
```

```r
# grepl(): Searches for patterns and returns a logical vector indicating matches
grepl("R", text)
```

```
[1] FALSE  TRUE FALSE
```

```r
# sub(): Replaces the first occurrence of a pattern in a string
sub("great", "awesome", text)
```

```
[1] "Hello, World!" "R is awesome"  "Data Science"
```

```r
# gsub(): Replaces all occurrences of a pattern in a string
gsub(" ", "_", text)
```

```
[1] "Hello,_World!" "R_is_great"    "Data_Science"
```

43.3 String Splitting and Joining

```
# strsplit(): Splits a string into substrings based on a specified delimiter
strsplit(text, split = " ")
```

```
[[1]]
[1] "Hello," "World!"

[[2]]
[1] "R"     "is"    "great"

[[3]]
[1] "Data"    "Science"
```

```
# unlist(): Converts a list to a vector (useful after strsplit)
unlist(strsplit(text, split = " "))
```

```
[1] "Hello,"  "World!"  "R"       "is"      "great"   "Data"    "Science"
```

43.4 Regular Expressions

```
# regexpr(): Finds the position of the first match of a pattern in a string
regexpr("R", text)
```

```
[1] -1  1 -1
attr(,"match.length")
[1] -1  1 -1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
# gregexpr(): Finds the positions of all matches of a pattern in a string
gregexpr(" ", text)
```

```
[[1]]
[1] 7
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE

[[2]]
[1] 2 5
attr(,"match.length")
[1] 1 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE

[[3]]
[1] 5
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
# regmatches(): Extracts the matched substrings based on the positions returned by regexpr or
regmatches(text, gregexpr(" ", text))
```

```
[[1]]
[1] " "

[[2]]
[1] " " " "

[[3]]
[1] " "
```

43.5 Example Usage

```
# Example: Convert text to lowercase and replace spaces with underscores
cleaned_text <- tolower(gsub(" ", "_", text))
cleaned_text
```

```
[1] "hello,_world!" "r_is_great"    "data_science"
```

These functions provide a solid foundation for string manipulation in R. For more advanced string operations, you # using the `stringr` package, which offers a more consistent and user-friendly interface for string handling.

44 Overall differences

We'll begin with a lookup table between the most important stringr functions and their base R equivalents.

```
library(stringr)
data_stringr_base_diff <- tibble::tribble(
  ~stringr,                                          ~base_r,
  "str_detect(string, pattern)",                     "grepl(pattern, x)",
  "str_dup(string, times)",                          "strrep(x, times)",
  "str_extract(string, pattern)",                    "regmatches(x, m = regexpr(pattern, text)
  "str_extract_all(string, pattern)",                "regmatches(x, m = gregexpr(pattern, text
  "str_length(string)",                              "nchar(x)",
  "str_locate(string, pattern)",                     "regexpr(pattern, text)",
  "str_locate_all(string, pattern)",                 "gregexpr(pattern, text)",
  "str_match(string, pattern)",                      "regmatches(x, m = regexec(pattern, text)
  "str_order(string)",                               "order(...)",
  "str_replace(string, pattern, replacement)",       "sub(pattern, replacement, x)",
  "str_replace_all(string, pattern, replacement)", "gsub(pattern, replacement, x)",
  "str_sort(string)",                                "sort(x)",
  "str_split(string, pattern)",                      "strsplit(x, split)",
  "str_sub(string, start, end)",                     "substr(x, start, stop)",
  "str_subset(string, pattern)",                     "grep(pattern, x, value = TRUE)",
  "str_to_lower(string)",                            "tolower(x)",
  "str_to_title(string)",                            "tools::toTitleCase(text)",
  "str_to_upper(string)",                            "toupper(x)",
  "str_trim(string)",                                "trimws(x)",
  "str_which(string, pattern)",                      "grep(pattern, x)",
  "str_wrap(string)",                                "strwrap(x)"
)

# create MD table, arranged alphabetically by stringr fn name
data_stringr_base_diff |>
  dplyr::mutate(dplyr::across(
      .cols = everything(),
      .fns = ~ paste0("`", .x, "`"))
  ) |>
```

stringr	base R
`str_detect(string, pattern)`	`grepl(pattern, x)`
`str_dup(string, times)`	`strrep(x, times)`
`str_extract(string, pattern)`	`regmatches(x, m = regexpr(pattern, text`
`str_extract_all(string, pattern)`	`regmatches(x, m = gregexpr(pattern, tex`
`str_length(string)`	`nchar(x)`
`str_locate(string, pattern)`	`regexpr(pattern, text)`
`str_locate_all(string, pattern)`	`gregexpr(pattern, text)`
`str_match(string, pattern)`	`regmatches(x, m = regexec(pattern, text`
`str_order(string)`	`order(...)`
`str_replace(string, pattern, replacement)`	`sub(pattern, replacement, x)`
`str_replace_all(string, pattern, replacement)`	`gsub(pattern, replacement, x)`
`str_sort(string)`	`sort(x)`
`str_split(string, pattern)`	`strsplit(x, split)`
`str_sub(string, start, end)`	`substr(x, start, stop)`
`str_subset(string, pattern)`	`grep(pattern, x, value = TRUE)`
`str_to_lower(string)`	`tolower(x)`
`str_to_title(string)`	`tools::toTitleCase(text)`
`str_to_upper(string)`	`toupper(x)`
`str_trim(string)`	`trimws(x)`
`str_which(string, pattern)`	`grep(pattern, x)`
`str_wrap(string)`	`strwrap(x)`

```
dplyr::arrange(stringr) |>
dplyr::rename(`base R` = base_r) |>
gt::gt() |>
gt::fmt_markdown(columns = everything()) |>
gt::tab_options(column_labels.font.weight = "bold")
```

Overall the main differences between base R and stringr are:

1. stringr functions start with `str_` prefix; base R string functions have no consistent naming scheme.
2. The order of inputs is usually different between base R and stringr. In base R, the `pattern` to match usually comes first; in stringr, the `string` to manupulate always comes first. This makes stringr easier to use in pipes, and with `lapply()` or `purrr::map()`.
3. Functions in stringr tend to do less, where many of the string processing functions in base R have multiple purposes.

4. The output and input of stringr functions has been carefully designed. For example, the output of `str_locate()` can be fed directly into `str_sub()`; the same is not true of `regexpr()` and `substr()`.

5. Base functions use arguments (like `perl`, `fixed`, and `ignore.case`) to control how the pattern is interpreted. To avoid dependence between arguments, stringr instead uses helper functions (like `fixed()`, `regex()`, and `coll()`).

Next we'll walk through each of the functions, noting the similarities and important differences. These examples are adapted from the stringr documentation and here they are contrasted with the analogous base R operations.

45 Detect matches

45.1 `str_detect()`: Detect the presence or absence of a pattern in a string

Suppose you want to know whether each word in a vector of fruit names contains an "a".

```
fruit <- c("apple", "banana", "pear", "pineapple")

# base
grepl(pattern = "a", x = fruit)
```

```
[1] TRUE TRUE TRUE TRUE
```

```
#stringr
stringr::str_detect(fruit, pattern = "a")
```

```
[1] TRUE TRUE TRUE TRUE
```

In base you would use `grepl()` (see the "l" and think logical) while in stringr you use `str_detect()` (see the verb "detect" and think of a yes/no action).

45.2 `str_which()`: Find positions matching a pattern

Now you want to identify the positions of the words in a vector of fruit names that contain an "a".

```
# base
grep(pattern = "a", x = fruit)
```

```
[1] 1 2 3 4
```

```
# stringr
str_which(fruit, pattern = "a")
```

```
[1] 1 2 3 4
```

In base you would use `grep()` while in stringr you use `str_which()` (by analogy to `which()`).

45.3 `str_count()`: Count the number of matches in a string

How many "a"s are in each fruit?

```
# base
loc <- gregexpr(pattern = "a", text = fruit, fixed = TRUE)
sapply(loc, function(x) length(attr(x, "match.length")))
```

```
[1] 1 3 1 1
```

```
# stringr
str_count(fruit, pattern = "a")
```

```
[1] 1 3 1 1
```

This information can be gleaned from `gregexpr()` in base, but you need to look at the `match.length` attribute as the vector uses a length-1 integer vector (`-1`) to indicate no match.

45.4 `str_locate()`: Locate the position of patterns in a string

Within each fruit, where does the first "p" occur? Where are all of the "p"s?

```
fruit3 <- c("papaya", "lime", "apple")

# base
str(gregexpr(pattern = "p", text = fruit3))
```

```
List of 3
 $ : int [1:2] 1 3
  ..- attr(*, "match.length")= int [1:2] 1 1
  ..- attr(*, "index.type")= chr "chars"
  ..- attr(*, "useBytes")= logi TRUE
 $ : int -1
  ..- attr(*, "match.length")= int -1
  ..- attr(*, "index.type")= chr "chars"
  ..- attr(*, "useBytes")= logi TRUE
 $ : int [1:2] 2 3
  ..- attr(*, "match.length")= int [1:2] 1 1
  ..- attr(*, "index.type")= chr "chars"
  ..- attr(*, "useBytes")= logi TRUE
```

```
# stringr
str_locate(fruit3, pattern = "p")
```

```
     start end
[1,]     1   1
[2,]    NA  NA
[3,]     2   2
```

```
str_locate_all(fruit3, pattern = "p")
```

```
[[1]]
     start end
[1,]     1   1
[2,]     3   3

[[2]]
     start end

[[3]]
     start end
[1,]     2   2
[2,]     3   3
```

45.5 `str_subset()`: Keep strings matching a pattern, or find positions

We may want to retrieve strings that contain a pattern of interest:

```r
# base
grep(pattern = "g", x = fruit, value = TRUE)
```

```
character(0)
```

```r
# stringr
str_subset(fruit, pattern = "g")
```

```
character(0)
```

45.6 `str_extract()`: Extract matching patterns from a string

We may want to pick out certain patterns from a string, for example, the digits in a shopping list:

```r
shopping_list <- c("apples x4", "bag of flour", "10", "milk x2")

# base
matches <- regexpr(pattern = "\\d+", text = shopping_list) # digits
regmatches(shopping_list, m = matches)
```

```
[1] "4"  "10" "2"
```

```r
matches <- gregexpr(pattern = "[a-z]+", text = shopping_list) # words
regmatches(shopping_list, m = matches)
```

```
[[1]]
[1] "apples" "x"

[[2]]
[1] "bag"   "of"    "flour"

[[3]]
character(0)

[[4]]
[1] "milk" "x"
```

```r
# stringr
str_extract(shopping_list, pattern = "\\d+")
```

```
[1] "4"  NA   "10" "2"
```

```r
str_extract_all(shopping_list, "[a-z]+")
```

```
[[1]]
[1] "apples" "x"

[[2]]
[1] "bag"   "of"    "flour"

[[3]]
character(0)

[[4]]
[1] "milk" "x"
```

Base R requires the combination of `regexpr()` with `regmatches()`; but note that the strings without matches are dropped from the output. stringr provides `str_extract()` and `str_extract_all()`, and the output is always the same length as the input.

45.7 `str_match()`: Extract matched groups from a string

We may also want to extract groups from a string. Here I'm going to use the scenario from Section 14.4.3 in R for Data Science.

```r
head(sentences)
```

```
[1] "The birch canoe slid on the smooth planks."
[2] "Glue the sheet to the dark blue background."
[3] "It's easy to tell the depth of a well."
[4] "These days a chicken leg is a rare dish."
[5] "Rice is often served in round bowls."
[6] "The juice of lemons makes fine punch."
```

```r
noun <- "([A]a|[Tt]he) ([^ ]+)"

# base
matches <- regexec(pattern = noun, text = head(sentences))
do.call("rbind", regmatches(x = head(sentences), m = matches))
```

```
     [,1]         [,2]  [,3]
[1,] "The birch"  "The" "birch"
[2,] "the sheet"  "the" "sheet"
[3,] "the depth"  "the" "depth"
[4,] "The juice"  "The" "juice"
```

```r
# stringr
str_match(head(sentences), pattern = noun)
```

```
     [,1]         [,2]  [,3]
[1,] "The birch"  "The" "birch"
[2,] "the sheet"  "the" "sheet"
[3,] "the depth"  "the" "depth"
[4,] NA           NA    NA
[5,] NA           NA    NA
[6,] "The juice"  "The" "juice"
```

As for extracting the full match base R requires the combination of two functions, and inputs with no matches are dropped from the output.

46 Manage lengths

46.1 `str_length()`: The length of a string

To determine the length of a string, base R uses `nchar()` (not to be confused with `length()` which gives the length of vectors, etc.) while stringr uses `str_length()`.

```
# base
nchar(letters)
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
# stringr
str_length(letters)
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

There are some subtle differences between base and stringr here. `nchar()` requires a character vector, so it will return an error if used on a factor. `str_length()` can handle a factor input.

```
# base
nchar(factor("abc"))
```

```
Error in nchar(factor("abc")): 'nchar()' requires a character vector
```

```
# stringr
str_length(factor("abc"))
```

```
[1] 3
```

Note that "characters" is a poorly defined concept, and technically both `nchar()` and `str_length()` returns the number of code points. This is usually the same as what you'd consider to be a charcter, but not always:

```
x <- c("\u00fc", "u\u0308")
x
```

```
[1] "ü" "ü"
```

```
nchar(x)
```

```
[1] 1 2
```

```
str_length(x)
```

```
[1] 1 2
```

46.2 `str_pad()`: Pad a string

To pad a string to a certain width, use stringr's `str_pad()`. In base R you could use `sprintf()`, but unlike `str_pad()`, `sprintf()` has many other functionalities.

```
# base
sprintf("%30s", "Sriram")
```

```
[1] "                       Sriram"
```

```
sprintf("%-30s", "Sriram")
```

```
[1] "Sriram                       "
```

```
# "both" is not as straightforward

# stringr
rbind(
  str_pad("Sriram", 30, "left"),
  str_pad("Sriram", 30, "right"),
  str_pad("Sriram", 30, "both")
)
```

```
     [,1]
[1,] "                       Sriram"
[2,] "Sriram                       "
[3,] "            Sriram            "
```

46.3 `str_trunc()`: Truncate a character string

The stringr package provides an easy way to truncate a character string: `str_trunc()`. Base R has no function to do this directly.

```
x <- "This string is moderately long"

# stringr
rbind(
  str_trunc(x, 20, "right"),
  str_trunc(x, 20, "left"),
  str_trunc(x, 20, "center")
)
```

```
     [,1]
[1,] "This string is mo..."
[2,] "...s moderately long"
[3,] "This stri...ely long"
```

46.4 `str_trim()`: Trim whitespace from a string

Similarly, stringr provides `str_trim()` to trim whitespace from a string. This is analogous to base R's `trimws()` added in R 3.3.0.

```
# base
trimws(" String with trailing and leading white space\t")
```

```
[1] "String with trailing and leading white space"
```

```
trimws("\n\nString with trailing and leading white space\n\n")
```

```
[1] "String with trailing and leading white space"
```

```
# stringr
str_trim(" String with trailing and leading white space\t")
```

```
[1] "String with trailing and leading white space"
```

```
str_trim("\n\nString with trailing and leading white space\n\n")
```

```
[1] "String with trailing and leading white space"
```

The stringr function `str_squish()` allows for extra whitespace within a string to be trimmed (in contrast to `str_trim()` which removes whitespace at the beginning and/or end of string). In base R, one might take advantage of `gsub()` to accomplish the same effect.

```
# stringr
str_squish(" String with trailing, middle,   and leading white space\t")
```

```
[1] "String with trailing, middle, and leading white space"
```

```
str_squish("\n\nString with excess, trailing and leading white space\n\n")
```

```
[1] "String with excess, trailing and leading white space"
```

46.5 `str_wrap()`: Wrap strings into nicely formatted paragraphs

`strwrap()` and `str_wrap()` use different algorithms. `str_wrap()` uses the famous Knuth-Plass algorithm.

```
gettysburg <- "Four score and seven years ago our fathers brought forth on this continent, a

# base
cat(strwrap(gettysburg, width = 60), sep = "\n")
```

```
Four score and seven years ago our fathers brought forth on
this continent, a new nation, conceived in Liberty, and
dedicated to the proposition that all men are created
equal.
```

```
# stringr
cat(str_wrap(gettysburg, width = 60), "\n")
```

```
Four score and seven years ago our fathers brought forth
on this continent, a new nation, conceived in Liberty, and
dedicated to the proposition that all men are created equal.
```

Note that `strwrap()` returns a character vector with one element for each line; `str_wrap()` returns a single string containing line breaks.

47 Mutate strings

47.1 `str_replace()`: Replace matched patterns in a string

To replace certain patterns within a string, stringr provides the functions `str_replace()` and `str_replace_all()`. The base R equivalents are `sub()` and `gsub()`. Note the difference in default input order again.

```
fruits <- c("apple", "banana", "pear", "pineapple")

# base
sub("[aeiou]", "-", fruits)
```

```
[1] "-pple"     "b-nana"    "p-ar"      "p-neapple"
```

```
gsub("[aeiou]", "-", fruits)
```

```
[1] "-ppl-"     "b-n-n-"    "p--r"      "p-n--ppl-"
```

```
# stringr
str_replace(fruits, "[aeiou]", "-")
```

```
[1] "-pple"     "b-nana"    "p-ar"      "p-neapple"
```

```
str_replace_all(fruits, "[aeiou]", "-")
```

```
[1] "-ppl-"     "b-n-n-"    "p--r"      "p-n--ppl-"
```

47.2 case: Convert case of a string

Both stringr and base R have functions to convert to upper and lower case. Title case is also provided in stringr.

```
dog <- "The quick brown dog"

# base
toupper(dog)
```

```
[1] "THE QUICK BROWN DOG"
```

```
tolower(dog)
```

```
[1] "the quick brown dog"
```

```
tools::toTitleCase(dog)
```

```
[1] "The Quick Brown Dog"
```

```
# stringr
str_to_upper(dog)
```

```
[1] "THE QUICK BROWN DOG"
```

```
str_to_lower(dog)
```

```
[1] "the quick brown dog"
```

```
str_to_title(dog)
```

```
[1] "The Quick Brown Dog"
```

In stringr we can control the locale, while in base R locale distinctions are controlled with global variables. Therefore, the output of your base R code may vary across different computers with different global settings.

```
# stringr
str_to_upper("i") # English
```

```
[1] "I"
```

```r
str_to_upper("i", locale = "tr") # Turkish
```

```
[1] "İ"
```

48 Join and split

48.1 `str_flatten()`: Flatten a string

If we want to take elements of a string vector and collapse them to a single string we can use the `collapse` argument in `paste()` or use stringr's `str_flatten()`.

```
# base
paste0(letters, collapse = "-")
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

```
# stringr
str_flatten(letters, collapse = "-")
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

The advantage of `str_flatten()` is that it always returns a vector the same length as its input; to predict the return length of `paste()` you must carefully read all arguments.

48.2 `str_dup()`: duplicate strings within a character vector

To duplicate strings within a character vector use `strrep()` (in R 3.3.0 or greater) or `str_dup()`:

```
fruit <- c("apple", "pear", "banana")

# base
strrep(fruit, 2)
```

```
[1] "appleapple"   "pearpear"     "bananabanana"
```

```
strrep(fruit, 1:3)
```

```
[1] "apple"               "pearpear"            "bananabananabanana"
```

```
# stringr
str_dup(fruit, 2)
```

```
[1] "appleapple"   "pearpear"     "bananabanana"
```

```
str_dup(fruit, 1:3)
```

```
[1] "apple"               "pearpear"            "bananabananabanana"
```

48.3 `str_split()`: Split up a string into pieces

To split a string into pieces with breaks based on a particular pattern match stringr uses `str_split()` and base R uses `strsplit()`. Unlike most other functions, `strsplit()` starts with the character vector to modify.

```
fruits <- c(
  "apples and oranges and pears and bananas",
  "pineapples and mangos and guavas"
)
# base
strsplit(fruits, " and ")
```

```
[[1]]
[1] "apples"  "oranges" "pears"   "bananas"

[[2]]
[1] "pineapples" "mangos"     "guavas"
```

```
# stringr
str_split(fruits, " and ")
```

```
[[1]]
[1] "apples"  "oranges" "pears"   "bananas"

[[2]]
[1] "pineapples" "mangos"     "guavas"
```

The stringr package's `str_split()` allows for more control over the split, including restricting the number of possible matches.

```
# stringr
str_split(fruits, " and ", n = 3)
```

```
[[1]]
[1] "apples"            "oranges"           "pears and bananas"

[[2]]
[1] "pineapples" "mangos"     "guavas"
```

```
str_split(fruits, " and ", n = 2)
```

```
[[1]]
[1] "apples"                       "oranges and pears and bananas"

[[2]]
[1] "pineapples"        "mangos and guavas"
```

48.4 `str_glue()`: Interpolate strings

It's often useful to interpolate varying values into a fixed string. In base R, you can use `sprintf()` for this purpose; stringr provides a wrapper for the more general purpose glue package.

```
name <- "Fred"
age <- 50
anniversary <- as.Date("1991-10-12")

# base
sprintf(
  "My name is %s my age next year is %s and my anniversary is %s.",
```

```
  name,
  age + 1,
  format(anniversary, "%A, %B %d, %Y")
)
```

```
[1] "My name is Fred my age next year is 51 and my anniversary is Saturday, October 12, 1991
```

```
# stringr
str_glue(
  "My name is {name}, ",
  "my age next year is {age + 1}, ",
  "and my anniversary is {format(anniversary, '%A, %B %d, %Y')}."
)
```

```
My name is Fred, my age next year is 51, and my anniversary is Saturday, October 12, 1991.
```

49 Order strings

49.1 `str_order()`: Order or sort a character vector

Both base R and stringr have separate functions to order and sort strings.

```
# base
order(letters)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
[26] 26
```

```
sort(letters)
```

```
 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
[20] "t" "u" "v" "w" "x" "y" "z"
```

```
# stringr
str_order(letters)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
[26] 26
```

```
str_sort(letters)
```

```
 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
[20] "t" "u" "v" "w" "x" "y" "z"
```

Some options in `str_order()` and `str_sort()` don't have analogous base R options. For example, the stringr functions have a `locale` argument to control how to order or sort. In base R the locale is a global setting, so the outputs of `sort()` and `order()` may differ across different computers. For example, in the Norwegian alphabet, å comes after z:

```r
x <- c("å", "a", "z")
str_sort(x)
```

```
[1] "a" "å" "z"
```

```r
str_sort(x, locale = "no")
```

```
[1] "a" "z" "å"
```

The stringr functions also have a `numeric` argument to sort digits numerically instead of treating them as strings.

```r
# stringr
x <- c("100a10", "100a5", "2b", "2a")
str_sort(x)
```

```
[1] "100a10" "100a5"  "2a"     "2b"
```

```r
str_sort(x, numeric = TRUE)
```

```
[1] "2a"     "2b"     "100a5"  "100a10"
```

```r
library(dplyr)
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
ae <- haven::read_sas("./data/sdtm/ae.sas7bdat")
ae1 <- ae |>
  select (USUBJID,AEDECOD,AEBODSYS)

#find function in sas, str_detect

ae2 <- ae1 |>
  filter(str_detect(AEDECOD,"HER"))

ae2 <- ae1 |>
  filter(str_detect(AEDECOD,"^HIATUS ")) # start of the string

ae2 <- ae1 |>
  filter(str_detect(AEDECOD,"HERNIA$")) # end of the string

#substr in sas

ae2 <- ae1 |>
  mutate(newvar1=str_sub(AEDECOD,1,6),
         newvar2=str_sub(AEDECOD,2,6),
         newvar3=str_sub(AEDECOD,-3),
         newvar4=str_length(AEDECOD))

#cat function in sas , Str_c in r

ae2 <- ae1 |>
  mutate(newvar1=str_c(AEDECOD,AEBODSYS,sep="/"),
         newvar2=paste(AEDECOD,AEBODSYS,sep="/"))

# scan function in sas, word in r

ae2 <- ae1 |>
  mutate(newvar1=word(AEDECOD,1),
         newvar2=word(AEDECOD,2))

# upper & lower case

ae2 <- ae1 |>
  mutate(newvar1=str_to_lower(AEDECOD),
         newvar2=str_to_upper(AEDECOD),
         newvar3=str_to_title(AEDECOD),
         newvar4=str_to_sentence(AEDECOD))
```

```
#str_trim

a <- "  this is    my String   "
b <- str_trim(a)
c <- str_replace_all(a," "," ")
d <- str_squish(a)
```

50 date and time functions

50.0.1 Date and Time Base Functions in R

Date and time manipulation is a common task in data analysis. R provides several base functions to work with dates and times. Here are some of the most commonly used base R functions for handling date and time data:

```
# Get the current date
current_date <- Sys.Date()
print(current_date)
```

```
[1] "2025-11-14"
```

```
# Get the current date and time
current_datetime <- Sys.time()
print(current_datetime)
```

```
[1] "2025-11-14 10:28:12 UTC"
```

50.0.2 Creating Date and Time Objects

50.0.3 Year Formats

Code	Meaning	Example (`format(dt, ...)`)
`%Y`	4-digit year	`"2025"`
`%y`	2-digit year (00–99)	`"25"`
`%C`	Century (year / 100, 00–99)	`"20"` (for 2025; OS-dep.)

50.0.4 Month Formats

Code	Meaning	Example
%m	Month number (01–12)	"11"
%b	Abbrev. month name (locale)	"Nov"
%B	Full month name (locale)	"November"
%h	Same as %b (abbrev. month name)	"Nov"

50.0.5 Day Formats

Code	Meaning	Example
%d	Day of month (01–31)	"14"
%e	Day of month (1–31, padded with space)	"14" (or " 7" for 7th; OS-dep.)
%j	Day of year (001–366)	"318"

50.0.6 Weekday Formats

Code	Meaning	Example
%a	Abbrev. weekday name (locale)	"Fri"
%A	Full weekday name (locale)	"Friday"
%w	Weekday number (0–6, **0 = Sunday**)	"5"
%u	ISO weekday number (1–7, **1 = Monday**) (OS-dep.)	"5" (Friday)

50.0.7 Time Formats

Code	Meaning	Example
%H	Hour (00–23)	"14"
%I	Hour (01–12)	"02"
%M	Minute (00–59)	"30"
%S	Second (00–61, allows for leap)	"05"
%p	AM/PM indicator (locale)	"PM"

###Locale-dependent “full date/time” codes

###These depend on your system locale (language + region):

Code	Meaning	Example
%c	Date and time representation	"Fri Nov 14 16:05:30 2025"
%x	Date representation	"11/14/25" (varies by locale)
%X	Time representation	"16:05:30"

```
# Create a Date object
dt <- as.Date("2025-11-14")
# Create a POSIXct object (date-time)
dt <- as.POSIXct("2025-11-14 16:05:30")
# Format examples
format(dt, "%Y-%m-%d")  # e.g. "2025-11-14"
```

```
[1] "2025-11-14"
```

```
format(dt, "%m/%d/%y")  # e.g. "11/14/25"
```

```
[1] "11/14/25"
```

```
format(dt, "%B %d, %Y") # e.g. "November 14, 2025"
```

```
[1] "November 14, 2025"
```

```
format(dt, "%c")  # e.g. "Fri Nov 14 16:05:30 2025"
```

```
[1] "Fri Nov 14 16:05:30 2025"
```

```
format(dt, "%x")  # e.g. "11/14/25" (depends on locale)
```

```
[1] "11/14/25"
```

```
format(dt, "%X")  # e.g. "16:05:30"
```

```
[1] "16:05:30"
```

SAS informat	R parsing
`ddmmyy10.`	`as.Date(x, "%d/%m/%Y")`
`ddmmyyd10.`	`as.Date(x, "%d-%m-%Y")`
`ddmmyyp10.`	`as.Date(x, "%d.%m.%Y")`
`mmddyy10.`	`as.Date(x, "%m/%d/%Y")`
`date9.`	`as.Date(x, "%d%b%Y")`
`julian7.`	`as.Date(strptime(x, "%Y%j"))`
`time8.`	`as.POSIXct(x, "%H:%M:%S")`
`time10.`	`as.POSIXct(x, "%I:%M:%S%p")` (if am/pm)
`datetime18.`	`as.POSIXct(x, "%d%b%Y:%H:%M:%S")`
`datetime20.`	`as.POSIXct(x, "%d%b%Y:%I:%M:%S%p")`

SAS format	Meaning	R equivalent
`worddate18.`	Month date, year (e.g., February 23, 2003)	`format(d, "%B %d, %Y")`
`weekdate24.`	Weekday, month date, year	`format(d, "%A, %B %d, %Y")`
`year.`	numeric year	`as.integer(format(d, "%Y"))`
`month.`	month number (1–12)	`as.integer(format(d, "%m"))`
`day.`	day of month	`as.integer(format(d, "%d"))`
`weekday.`	day of week (1–7)	`as.integer(format(d, "%u"))` (Mon = 1)

```r
# Convert a character string to a Date object
date_string <- "2023-10-01"
date_object <- as.Date(date_string)
print(date_object)
```

```
[1] "2023-10-01"
```

```r
# Convert a character string to a POSIXct object (date-time)
datetime_string <- "2023-10-01 12:34:56"
datetime_object <- as.POSIXct(datetime_string)
print(datetime_object)
```

```
[1] "2023-10-01 12:34:56 UTC"
```

```r
# Extract components from a Date object
year <- format(date_object, "%Y")
month <- format(date_object, "%m")
day <- format(date_object, "%d")
print(paste("Year:", year, "Month:", month, "Day:", day))
```

```
[1] "Year: 2023 Month: 10 Day: 01"
```

```r
# Extract components from a POSIXct object
hour <- format(datetime_object, "%H")
minute <- format(datetime_object, "%M")
second <- format(datetime_object, "%S")
print(paste("Hour:", hour, "Minute:", minute, "Second:", second))
```

```
[1] "Hour: 12 Minute: 34 Second: 56"
```

```r
# Perform date arithmetic
tomorrow <- current_date + 1
yesterday <- current_date - 1
print(paste("Tomorrow:", tomorrow, "Yesterday:", yesterday))
```

```
[1] "Tomorrow: 2025-11-15 Yesterday: 2025-11-13"
```

```r
# Calculate the difference between two dates
date_diff <- as.numeric(difftime(current_date, date_object, units = "days"))
print(paste("Difference in days:", date_diff))
```

```
[1] "Difference in days: 775"
```

```r
# Format a Date object to a specific string format
formatted_date <- format(current_date, "%B %d, %Y")
print(formatted_date)
```

```
[1] "November 14, 2025"
```

```r
# Format a POSIXct object to a specific string format
formatted_datetime <- format(current_datetime, "%Y-%m-%d %H:%M:%S")
print(formatted_datetime)
```

```
[1] "2025-11-14 10:28:12"
```

These functions allow you to create, manipulate, and format date and time data in R. For more advanced date and time handling, you might consider using the `lubridate` package, which provides a more user-friendly interface for working with dates and times.

A practical, CDISC-friendly guide to working with **dates/times** and implementing **partial date imputation** in R using **base R** and **lubridate**.

50.1 1. Date & Time Basics in R

50.1.1 1.1 Core classes

- **Date**: calendar dates without time.
- **POSIXct**: seconds since 1970-01-01 (compact, good for storage/arithmetic).
- **POSIXlt**: list of components (year, mon, mday, hour, min, sec) — convenient to extract parts.

50.1.2 1.2 Converting numerics to Date (origins matter)

Different systems use different **origins** for numeric dates: - Excel: 1900-01-01 (beware 1900 leap-year bug in older Excels) - SAS: 1960-01-01 - R: 1970-01-01

```
num_dates <- c(0, 159, 464, 15674)

as.Date(num_dates, origin = "1960-01-01")  # SAS origin
```

```
[1] "1960-01-01" "1960-06-08" "1961-04-09" "2002-11-30"
```

```
as.Date(num_dates, origin = "1970-01-01")  # R origin
```

```
[1] "1970-01-01" "1970-06-09" "1971-04-10" "2012-11-30"
```

```
as.Date(num_dates, origin = "1900-01-01")  # Excel origin
```

```
[1] "1900-01-01" "1900-06-09" "1901-04-10" "1942-12-01"
```

50.1.3 1.3 Custom date formats

```
x1 <- c("01/05/1965", "08/16/1975")
as.Date(x1, "%m/%d/%Y")

x2 <- c("05-01-1965", "16-08-1975")
as.Date(x2, "%d-%m-%Y")

x3 <- c("01MAY1965", "16AUG1975")
as.Date(x3, "%d%b%Y")

# Unknown format? Try multiple patterns
x4 <- c("01MAY1965MORNING", "16AUGUST1975")
as.Date(x4, tryFormats = c("%d%b%Y", "%d%B%Y"))
```

50.1.4 1.4 Datetime strings (ISO 8601 & time zones)

```
x <- c("2015-10-19T10:15", "2018-12-19T23:05")
# Keep the literal 'T' in the format string
as.POSIXct(x, format = "%Y-%m-%dT%H:%M", tz = "UTC")

# Extract parts
xt <- as.POSIXlt(x, format = "%Y-%m-%dT%H:%M")
format(xt, "%Y-%m-%d")  # date part
format(xt, "%H:%M")     # time part
```

50.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic

Install & load: `install.packages("lubridate"); library(lubridate)`

50.2.1 2.1 Intuitive parsers

- `ymd()`, `mdy()`, `dmy()` for dates
- `ymd_hms()`, `ymd_hm()`, `ymd_h()` for date-times

```
ymd("20210529")
dmy("29/05/2021")
ymd_hm("2019/01/19T17:15")  # delimiter agnostic
```

50.2.2 2.2 Accessors and rounding

```
x <- ymd_hms("2009-08-03 12:01:59")
year(x); month(x); mday(x); wday(x); hour(x); minute(x); second(x)

floor_date(x, "month")   # 2009-08-01
ceiling_date(x, "month") # 2009-09-01
round_date(x, "month")    # 2009-08-01
rollback(x)               # last day of previous month
```

50.2.3 2.3 Timespans

- **Durations**: exact seconds, e.g., `ddays(2)`
- **Periods**: human units, e.g., `months(1)`
- **Intervals**: start–end pair, e.g., `interval(start, end)`

```
start <- ymd("2015-10-19")
end   <- ymd("2018-12-19")
int   <- interval(start, end)
as.duration(int)       # exact seconds
as.period(int)         # years/months/days
```

50.2.4 3 ADaM-Style Partial Date Imputation (Start & End)

Imputation rules vary by study; always follow the SAP. A common pattern:

50.2.5 Start date (`AESTDTC`)

- `YYYY-MM-DD` → use as is
- `YYYY-MM` → impute **day = 01** (first of month)
- `YYYY` → impute **month = 01**, **day = 01** (`01-Jan`)

50.2.6 End date (AEENDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **last day of that month**
- YYYY → impute 31-Dec

50.2.7 Censoring by death / last alive date

Additionally, bound AEENDT by death / last alive date (when present):

- CENSOR = coalesce(DTHDT, LASTALVDT)
- AEENDT = min(imputed_end, CENSOR)

50.2.8 Imputation flags

Add flags:

- AESTDTF, AEENDTF:
  - "D" → day imputed
  - "M,D" → month & day imputed
- Add "C" if we had to **censor the end date** at DTHDT / LASTALVDT.

```
library(dplyr, warn.conflicts = FALSE)
library(stringr, warn.conflicts = FALSE)
library(lubridate, warn.conflicts = FALSE)

df <- data.frame(
  USUBJID    = sprintf("01-001-%03d", 1:6),
  AESTDTC    = c("2015-10-19", "2015-10", "2015", "2019-02", "2020-02", "2018-12-31"),
  AEENDTC    = c("2018-12-19", "2018-10", "2017-05-29", "2019-02", "2020-02", "2020"),
  RANDDT     = ymd(c("2015-10-01", "2015-10-01", "2015-01-10", "2019-01-15", "2020-02-10", "
  DTHDT      = ymd(c(NA, NA, NA, NA, "2020-08-03", NA)),
  LASTALVDT  = ymd(c(NA, NA, NA, NA, NA, "2020-12-15"))
)
```

```r
df_imp <- df |>
  mutate(
    # Length of partial date strings
    AESTDTC_LEN = str_length(AESTDTC),
    AEENDTC_LEN = str_length(AEENDTC),
    ## --- AE start date imputation (minimum: first possible date) ---
    AESTDT = case_when(
      is.na(AESTDTC) ~ as.Date(NA),
      AESTDTC_LEN == 10 ~ ymd(AESTDTC, quiet = TRUE),
      AESTDTC_LEN == 7  ~ ymd(str_c(AESTDTC, "-01"), quiet = TRUE),
      AESTDTC_LEN == 4  ~ ymd(str_c(AESTDTC, "-01-01"), quiet = TRUE),
      TRUE ~ as.Date(NA)
    ),
    AESTDTF = case_when(
      is.na(AESTDTC) ~ NA_character_,
      AESTDTC_LEN == 10 ~ "",
      AESTDTC_LEN == 7  ~ "D",
      AESTDTC_LEN == 4  ~ "M,D",
      TRUE ~ "UNK"
    ),
    ## --- AE end date imputation (maximum: latest possible date) ---
    AEENDT_RAW = case_when(
      is.na(AEENDTC) ~ as.Date(NA),
      AEENDTC_LEN == 10 ~ ymd(AEENDTC, quiet = TRUE),
      AEENDTC_LEN == 7  ~ ymd(str_c(AEENDTC, "-01"), quiet = TRUE),
      AEENDTC_LEN == 4  ~ ymd(str_c(AEENDTC, "-12-31"), quiet = TRUE),
      TRUE ~ as.Date(NA)
    ),
    AEENDTF_RAW = case_when(
      is.na(AEENDTC) ~ NA_character_,
      AEENDTC_LEN == 10 ~ "",
      AEENDTC_LEN == 7  ~ "D",
      AEENDTC_LEN == 4  ~ "M,D",
      TRUE ~ "UNK"
    )
  ) %>%
  mutate(
    AEENDT_RAW = if_else(
      AEENDTC_LEN == 7 & !is.na(AEENDT_RAW),
      ceiling_date(AEENDT_RAW, "month") - days(1),
      AEENDT_RAW
    ),
```

```
    CENSOR = coalesce(DTHDT, LASTALVDT),
    AEENDT = case_when(
      !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR ~ CENSOR,
      TRUE ~ AEENDT_RAW
    ),
    AEENDTF = case_when(
      is.na(AEENDT) ~ AEENDTF_RAW,
      !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW == ""  ~ "C",
      !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW != "" ~ str_c(A
      TRUE ~ AEENDTF_RAW
    )
  )
```

50.3 4. Durations & Analysis Variables

50.3.1 4.1 AE duration (AEDUR) in days

51 Using difftime (base) or as.numeric on Date differences

```
AEDUR <- as.integer(out$AEENDT - out$ASTDT)
```

51.0.1 6.2 Handy lubridate parsers (subset)

- Dates: `ymd()`, `mdy()`, `dmy()`, `yq()`
- Datetimes: `ymd_hms()`, `ymd_hm()`, `ymd_h()`
- Times: `hms()`, `hm()`, `ms()`
- Accessors: `year()`, `month()`, `mday()`, `wday()`, `hour()`, `minute()`, `second()`, `date()`
- Rounding: `floor_date()`, `ceiling_date()`, `round_date()`, `rollback()`

---

52 Reading SAS Datasets (+ Cleaning)

```
library(haven)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(labelled)
```

We try to read a SAS dataset (e.g., SDTM DM). If not present, we **synthesize** an example.

```
dm_path <- "data/sdtm/dm.sas7bdat"

if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:10),
    ARM = rep(c("Placebo","Active"), length.out=10),
    AGE = c(55, 62, 47, 50, 71, 66, 45, 59, 53, 68),
    SEX = rep(c("M","F"), length.out=10)
  )
  message("Synthesized `dm` since data/sdtm/dm.sas7bdat was not found.")
}
str(dm)
```

```
tibble [306 x 25] (S3: tbl_df/tbl/data.frame)
 $ STUDYID : chr [1:306] "CDISCPILOT01" "CDISCPILOT01" "CDISCPILOT01" "CDISCPILOT01" ...
  ..- attr(*, "label")= chr "Study Identifier"
 $ DOMAIN  : chr [1:306] "DM" "DM" "DM" "DM" ...
  ..- attr(*, "label")= chr "Domain Abbreviation"
 $ USUBJID : chr [1:306] "01-701-1015" "01-701-1023" "01-701-1028" "01-701-
1033" ...
  ..- attr(*, "label")= chr "Unique Subject Identifier"
 $ SUBJID  : chr [1:306] "1015" "1023" "1028" "1033" ...
  ..- attr(*, "label")= chr "Subject Identifier for the Study"
 $ RFSTDTC : chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
  ..- attr(*, "label")= chr "Subject Reference Start Date/Time"
 $ RFENDTC : chr [1:306] "2014-07-02" "2012-09-02" "2014-01-14" "2014-04-
14" ...
  ..- attr(*, "label")= chr "Subject Reference End Date/Time"
 $ RFXSTDTC: chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
  ..- attr(*, "label")= chr "Date/Time of First Study Treatment"
 $ RFXENDTC: chr [1:306] "2014-07-02" "2012-09-01" "2014-01-14" "2014-03-
31" ...
  ..- attr(*, "label")= chr "Date/Time of Last Study Treatment"
 $ RFICDTC : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Date/Time of Informed Consent"
 $ RFPENDTC: chr [1:306] "2014-07-02T11:45" "2013-02-18" "2014-01-14T11:10" "2014-
09-15" ...
  ..- attr(*, "label")= chr "Date/Time of End of Participation"
 $ DTHDTC  : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Date/Time of Death"
 $ DTHFL   : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Subject Death Flag"
 $ SITEID  : chr [1:306] "701" "701" "701" "701" ...
  ..- attr(*, "label")= chr "Study Site Identifier"
 $ AGE     : num [1:306] 63 64 71 74 77 85 59 68 81 84 ...
  ..- attr(*, "label")= chr "Age"
 $ AGEU    : chr [1:306] "YEARS" "YEARS" "YEARS" "YEARS" ...
  ..- attr(*, "label")= chr "Age Units"
 $ SEX     : chr [1:306] "F" "M" "M" "M" ...
  ..- attr(*, "label")= chr "Sex"
 $ RACE    : chr [1:306] "WHITE" "WHITE" "WHITE" "WHITE" ...
  ..- attr(*, "label")= chr "Race"
 $ ETHNIC  : chr [1:306] "HISPANIC OR LATINO" "HISPANIC OR LATINO" "NOT HISPANIC OR LATINO" "
  ..- attr(*, "label")= chr "Ethnicity"
```

```
 $ ARMCD   : chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
  ..- attr(*, "label")= chr "Planned Arm Code"
 $ ARM     : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
  ..- attr(*, "label")= chr "Description of Planned Arm"
 $ ACTARMCD: chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
  ..- attr(*, "label")= chr "Actual Arm Code"
 $ ACTARM  : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
  ..- attr(*, "label")= chr "Description of Actual Arm"
 $ COUNTRY : chr [1:306] "USA" "USA" "USA" "USA" ...
  ..- attr(*, "label")= chr "Country"
 $ DMDTC   : chr [1:306] "2013-12-26" "2012-07-22" "2013-07-11" "2014-03-
10" ...
  ..- attr(*, "label")= chr "Date/Time of Collection"
 $ DMDY    : num [1:306] -7 -14 -8 -8 -7 -21 NA -9 -13 -7 ...
  ..- attr(*, "label")= chr "Study Day of Collection"
```

52.1 Handling Labels & Missing

```r
# Example: Convert blank strings "" to NA for character columns
convert_blanks_to_na <- function(x) {
  if (is.character(x)) x[x == ""] <- NA_character_
  x
}
dm <- dm |> mutate(across(where(is.character), convert_blanks_to_na))
```

52.2 Labelled to Factor (if needed)

```r
if (inherits(dm$SEX, "labelled")) {
  dm <- dm |> mutate(SEX = to_factor(SEX))
}
```

52.3 Common Cleaning

```r
dm <- dm |>
  mutate(
    AGEGR1 = cut(AGE, breaks=c(-Inf, 50, 65, Inf),
                 labels=c("<50","50-65", "65+"))
  )
```

53 Exercises

1. Read another SAS dataset (e.g., `sv.sas7bdat`) if available. If not, create a synthetic tibble.
2. Write a function to trim character whitespace for all character columns.
3. Make a clean factor for `ARM` with levels `Placebo < Active`.

54 Base R Functions & Apply Family

55 Common Utilities

```
x <- 1:10
sum(x); mean(x); sd(x); var(x); quantile(x)
```

```
[1] 55
```

```
[1] 5.5
```

```
[1] 3.02765
```

```
[1] 9.166667
```

```
   0%   25%   50%   75%  100%
 1.00  3.25  5.50  7.75 10.00
```

```
seq(0, 1, by=0.1); rep(5, times=3)
```

```
 [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
[1] 5 5 5
```

56 Apply Family

```
m <- matrix(1:9, nrow=3)
apply(m, 1, mean)  # row means
```

```
[1] 4 5 6
```

```
apply(m, 2, mean)  # col means
```

```
[1] 2 5 8
```

```
lst <- list(a=1:3, b=10:12)
sapply(lst, mean)   # simplifies result
```

```
 a  b
 2 11
```

```
mapply(sum, 1:3, 10:12)
```

```
[1] 11 13 15
```

57 Subsetting Essentials

```
df <- data.frame(id=1:3, val=c(10,20,30))
df[1, "val"]
```

```
[1] 10
```

```
df[df$val > 10, ]
```

```
  id val
2  2  20
3  3  30
```

58 Exercises

1. Use `apply` to get the max per column of a numeric matrix.
2. Write a base R snippet to compute IQR for each column of `mtcars`.
3. Compare `lapply` vs `sapply` in behavior on a list with mixed types.

59 Custom Functions & Validation

60 Writing Functions

```
safe_mean <- function(x, na.rm = TRUE) {
  stopifnot(is.numeric(x))
  mean(x, na.rm = na.rm)
}
safe_mean(c(1, 2, NA))
```

```
[1] 1.5
```

61 Error Handling

```
robust_divide <- function(a, b) {
  tryCatch(a / b,
           warning = function(w) NA_real_,
           error   = function(e) NA_real_)
}
robust_divide(10, 0)
```

```
[1] Inf
```

62 Unit Testing with testthat

Install once: `install.packages(c("testthat","devtools","usethis","roxygen2"))`

```
usethis::use_testthat()
usethis::use_test("safe_mean")
```

Create `tests/testthat/test-safe_mean.R`:

```
testthat::test_that("safe_mean works", {
  x <- c(1,2,NA)
  testthat::expect_equal(safe_mean(x), 1.5)
  testthat::expect_error(safe_mean("oops"))
})
```

```
Test passed with 2 successes.
```

63 Document with roxygen2

```
#' Compute a safe mean
#' @param x Numeric vector
#' @param na.rm Logical; remove NAs
#' @return Numeric scalar
#' @examples
#' safe_mean(c(1,2,NA))
#' @export
safe_mean <- function(x, na.rm = TRUE) {
  stopifnot(is.numeric(x))
  mean(x, na.rm = na.rm)
}
```

Run:

```
devtools::document()
```

64 Exercises

1. Write `winsorize(x, probs=c(0.05,0.95))` and test it.
2. Create `validate_columns(df, required=c("USUBJID","AGE"))` and add tests.
3. Add roxygen docs and build help pages.

65 R Package Development

65.1 Setup

```
install.packages(c("usethis","devtools","testthat","roxygen2","pkgdown"))
```

65.2 Create a Package

```
usethis::create_package("mypkg")
# In the new project:
usethis::use_mit_license("Your Name")
usethis::use_git()
usethis::use_github()   # optional
usethis::use_roxygen_md()
usethis::use_testthat()
usethis::use_package("dplyr")   # adds to DESCRIPTION
```

65.3 Add a Function

Create `R/safe_mean.R` and its tests (see previous chapter).

65.4 Build, Install, Check

```
devtools::document()
devtools::build()
devtools::install()
devtools::check()
```

65.5 Vignette & Website

```
usethis::use_vignette("intro")
usethis::use_pkgdown()
pkgdown::build_site()
```

Exercise: Package-ize a small utility set (`convert_blanks_to_na`, `validate_columns`, etc.) with docs and tests.

66 Git in RStudio (Setup & Auth)

66.1 One-Time Setup

- Install Git and ensure `git --version` works.
- In R:

```
usethis::use_git_config(user.name = "Your Name", user.email = "you@example.com")
```

66.2 Initialize Git for the Current Project

```
usethis::use_git()
```

66.3 Connect to GitHub

- Create a GitHub account.
- In R:

```
usethis::create_github_token()
gitcreds::gitcreds_set()  # paste token when prompted
usethis::use_github(protocol = "https")
```

Or set up SSH keys via RStudio (`Tools > Global Options > Git/SVN`).

66.4 Typical Workflow

1. Stage changes (Git pane in RStudio).
2. Commit with a clear message.
3. Push to origin (GitHub).

66.5 Remove Git from a Project (macOS/RStudio)

- In Finder/Terminal, delete the hidden `.git` folder in the project root (careful!).
- Or from Terminal at project root:

```
rm -rf .git
```

- Reopen project in RStudio; Git pane will disappear.

Exercises - Create a new repo for this Quarto course and push it. - Branch, make a change, open a Pull Request on GitHub.

67 Creating ADaM: ADSL from SDTM-like Inputs

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(tidyr)
library(lubridate)
```

```
Attaching package: 'lubridate'

The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

We simulate **minimal** SDTM-like `DM` and `EX` to illustrate ADSL creation. If available, replace with your own `data/sdtm/*.sas7bdat`.

```r
# DM
dm <- tibble::tibble(
  STUDYID = "XYZ123",
  USUBJID = sprintf("XYZ-%03d", 1:10),
  ARM = rep(c("Placebo","Active"), length.out=10),
  AGE = c(55, 62, 47, 50, 71, 66, 45, 59, 53, 68),
  SEX = rep(c("M","F"), length.out=10),
  RANDDT = as.Date("2025-01-15") + sample(0:20, 10, replace=TRUE)
)

# EX (first dose date)
ex <- tibble::tibble(
  USUBJID = dm$USUBJID,
  EXSTDTC = dm$RANDDT + sample(0:3, 10, replace=TRUE)
)
```

67.1 Build ADSL

```r
adsl <- dm |>
  left_join(ex, by="USUBJID") |>
  transmute(
    STUDYID, USUBJID,
    TRT01P = ARM,
    TRT01PN = as.integer(factor(ARM, levels=c("Placebo","Active"))),
    AGE, SEX,
    RANDDT,
    TRTSDT = EXSTDTC,
    TRT01A = TRT01P,          # assume planned == actual for demo
    TRT01AN = TRT01PN
  )
adsl
```

```
# A tibble: 10 x 10
   STUDYID USUBJID TRT01P  TRT01PN   AGE SEX   RANDDT     TRTSDT     TRT01A
   <chr>   <chr>   <chr>      <int> <dbl> <chr> <date>     <date>     <chr>
 1 XYZ123  XYZ-001 Placebo        1    55 M     2025-01-24 2025-01-24 Placebo
 2 XYZ123  XYZ-002 Active         2    62 F     2025-01-24 2025-01-25 Active
 3 XYZ123  XYZ-003 Placebo        1    47 M     2025-02-03 2025-02-03 Placebo
 4 XYZ123  XYZ-004 Active         2    50 F     2025-02-02 2025-02-04 Active
```

```
 5 XYZ123  XYZ-005 Placebo       1    71 M     2025-01-31 2025-02-03 Placebo
 6 XYZ123  XYZ-006 Active        2    66 F     2025-01-23 2025-01-25 Active
 7 XYZ123  XYZ-007 Placebo       1    45 M     2025-01-23 2025-01-25 Placebo
 8 XYZ123  XYZ-008 Active        2    59 F     2025-02-01 2025-02-03 Active
 9 XYZ123  XYZ-009 Placebo       1    53 M     2025-01-15 2025-01-15 Placebo
10 XYZ123  XYZ-010 Active        2    68 F     2025-01-31 2025-02-02 Active
# i 1 more variable: TRT01AN <int>
```

> Note: Real ADSL creation must follow **ADaM IG** (derive flags, dates, imputations, populations). This example is educational only.

Exercises 1. Add analysis populations (e.g., `SAFFL`, `FASFL`) based on simple rules. 2. Derive `AGEGR1` as `<65 / 65` and use ordered factor. 3. Add a treatment end date `TRTEDT` and compute treatment duration.

68 TLFs: Table, Figure, Listing

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(gt)
library(ggplot2)
library(survival)
```

We reuse `adsl` from the previous chapter (or synthesize if missing).

```
if (!exists("adsl")) {
  set.seed(123)
  adsl <- tibble::tibble(
    USUBJID = sprintf("XYZ-%03d", 1:60),
    TRT01P = sample(c("Placebo","Active"), 60, replace=TRUE),
    AGE = round(rnorm(60, 60, 8)),
    SEX = sample(c("M","F"), 60, replace=TRUE)
  )
}
```

[

Table 1. Baseline Characteristics by Treatment]
Table 1. Baseline Characteristics by Treatment

TRT01P	N	mean_age	sd_age	pct_female
Active	24	59.4	8.1	50.0
Placebo	36	61.7	5.6	61.1

68.1 Table 1: Baseline Characteristics by Treatment

```
tbl1 <- adsl |>
  group_by(TRT01P) |>
  summarise(
    N = dplyr::n(),
    mean_age = mean(AGE, na.rm=TRUE),
    sd_age = sd(AGE, na.rm=TRUE),
    pct_female = mean(SEX == "F")*100
  )

gt(tbl1) |>
  tab_header(title = "Table 1. Baseline Characteristics by Treatment") |>
  fmt_number(columns = c(mean_age, sd_age, pct_female), decimals = 1)
```

68.2 Figure: (Toy) Survival Curve

We simulate time-to-event data for illustration only.

```
set.seed(42)
n <- nrow(adsl)
adsl$time <- rexp(n, rate = ifelse(adsl$TRT01P=="Active", 0.08, 0.1))
adsl$status <- rbinom(n, 1, 0.7)
fit <- survival::survfit(survival::Surv(time, status) ~ TRT01P, data = adsl)

# Quick GGplot
ggsurv <- function(fit) {
  # rebuild data for plotting
  ss <- summary(fit)
```

```
  dd <- data.frame(
    time = ss$time,
    surv = ss$surv,
    strata = rep(names(fit$strata), fit$strata)
  )
  ggplot(dd, aes(x=time, y=surv, linetype=strata)) +
    geom_step() +
    labs(title="Kaplan-Meier (Toy Data)", x="Time", y="Survival Probability", linetype="Treat
    theme_minimal()
}
#ggsurv(fit)
```

68.3 Listing: Subject-Level Listing

```
lst <- adsl |>
  arrange(USUBJID) |>
  select(USUBJID, TRT01P, AGE, SEX) |>
  head(20)

gt(lst) |>
  tab_header(title = "Listing: First 20 Subjects")
```

Exercises 1. Format Table 1 to `N (mean ± SD)` for age. 2. Add risk table to the KM plot (use an extension like survminer outside of this minimal example). 3. Create a listing that includes population flags once you derive them.

[

Listing: First 20 Subjects] Listing: First 20 Subjects

USUBJID	TRT01P	AGE	SEX
XYZ-001	Placebo	63	F
XYZ-002	Placebo	58	M
XYZ-003	Placebo	67	M
XYZ-004	Active	67	M
XYZ-005	Placebo	67	M
XYZ-006	Active	66	F
XYZ-007	Active	64	F
XYZ-008	Active	60	M
XYZ-009	Placebo	58	M
XYZ-010	Placebo	57	F
XYZ-011	Active	54	F
XYZ-012	Active	58	M
XYZ-013	Active	50	M
XYZ-014	Placebo	77	F
XYZ-015	Active	70	F
XYZ-016	Placebo	51	M
XYZ-017	Active	57	M
XYZ-018	Placebo	56	F
XYZ-019	Placebo	66	M
XYZ-020	Placebo	59	F

69 Capstone: End-to-End Mini Workflow

This chapter ties everything together: **read data → derive ADSL → produce TLFs → render a report.**

69.1 Parameters

```
# You could parametrize paths via YAML; here we keep inline defaults.
dm_path <- "data/sdtm/dm.sas7bdat"
ex_path <- "data/sdtm/ex.sas7bdat"
```

69.2 1) Read (or Synthesize) SDTM

```
library(haven); library(dplyr); library(lubridate)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union


Attaching package: 'lubridate'
```

```
The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```
if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:60),
    ARM = sample(c("Placebo","Active"), 60, replace=TRUE),
    AGE = round(rnorm(60, 60, 8)),
    SEX = sample(c("M","F"), 60, replace=TRUE),
    RANDDT = as.Date("2025-01-15") + sample(0:40, 60, replace=TRUE)
  )
}
if (file.exists(ex_path)) {
  ex <- read_sas(ex_path)
} else {
  ex <- tibble::tibble(
    USUBJID = dm$USUBJID,
    EXSTDTC = dm$RANDDT + sample(0:3, nrow(dm), replace=TRUE)
  )
}
```

69.3 2) Derive ADSL (Minimal Demo)

```
adsl <- dm |>
  left_join(ex, by="USUBJID") |>
  mutate(
    TRT01P = ARM,
    TRT01PN = as.integer(factor(ARM, levels=c("Placebo","Active"))),
    TRT01A = TRT01P,
    TRT01AN = TRT01PN,
    SAFFL = "Y",          # demo only; define rules in real life
    FASFL = "Y"
  ) |>
  dplyr::select(STUDYID.x, USUBJID, TRT01P, TRT01PN, TRT01A, TRT01AN, AGE, SEX, EXSTDTC, SAFF
```

[

Table 1. Baseline by Treatment]

Table 1. Baseline by Treatment

Description of Planned Arm	N	mean_age	sd_age	pct_female
Placebo	226	75.04867	8.503715	60.61947
Screen Failure	52	75.09615	9.699928	69.23077
Xanomeline High Dose	184	74.01087	7.939656	48.36957
Xanomeline Low Dose	181	75.29834	8.277778	60.77348

69.4 3) TLFs

```
library(gt); library(ggplot2); library(survival)

tbl1 <- adsl |>
  group_by(TRT01P) |>
  summarise(N=n(),
            mean_age = mean(AGE), sd_age = sd(AGE),
            pct_female = mean(SEX=="F")*100)
tbl1_gt <- gt(tbl1) |> tab_header(title="Table 1. Baseline by Treatment")
tbl1_gt
```

```
set.seed(123)
adsl$time <- rexp(nrow(adsl), rate=ifelse(adsl$TRT01P=="Active", 0.08, 0.1))
adsl$status <- rbinom(nrow(adsl), 1, 0.7)
fit <- survfit(Surv(time, status) ~ TRT01P, data=adsl)
# reuse plotting function from prior chapter
ggsurv <- function(fit) {
  ss <- summary(fit)
  dd <- data.frame(time=ss$time, surv=ss$surv, strata=rep(names(fit$strata), fit$strata))
  ggplot(dd, aes(x=time, y=surv, linetype=strata)) + geom_step() + theme_minimal() +
    labs(title="KM Curve (Toy)", x="Time", y="Survival", linetype="Treatment")
}
#ggsurv(fit)
```

69.5 4) Save Outputs

```
# Example: Save Table 1 as PNG
#gtsave(tbl1_gt, "tlf-table1.png")
```

Challenge: Convert this chapter into a **parameterized report** (e.g., treatment subset or different cohort) and render multiple outputs.

70 Appendix: Tips, Profiles, .libPaths

70.1 Useful Profiles

Create `~/.Rprofile` to set options (be careful on shared systems):

```
options(
  repos = c(CRAN = "https://cloud.r-project.org"),
  scipen = 999
)
```

70.2 Custom Library Paths

```
# In .Rprofile or project-level .Rprofile
.libPaths(c("/path/to/Rlibs", .libPaths()))
```

70.3 Format vs formatC (quick recap)

```
x <- c(123.456, 0.00123456)
format(x, digits = 4)
```

```
[1] "1.235e+02" "1.235e-03"
```

```
format(x, nsmall = 2)
```

```
[1] "1.23456e+02" "1.23456e-03"
```

```
formatC(x, digits = 3, format = "f")
```

```
[1] "123.456" "0.001"
```

70.4 POSIXct vs POSIXlt

- **POSIXct**: seconds since epoch (numeric), compact, fast.
- **POSIXlt**: list-like with components (year, mon, mday…), easier to extract parts.

70.5 Recommended Packages

- `tidyverse`, `lubridate`, `janitor`, `gt`, `gtsummary`, `survival`, `broom`, `here`.
- Pharma/CDISC: `admiral`, `tlf/tern`, `pharmaverse` meta-packages (explore as you grow).

70.6 Short Glossary

- **SDTM**: Study Data Tabulation Model (FDA submission standard for raw domains).
- **ADaM**: Analysis Data Model (derived analysis-ready datasets).
- **TLF**: Tables, Listings, Figures for reporting.